IT Security Master Program
Department of Economic Informatics and Cybernetics
Faculty of Cybernetics, Statistics and Economic Informatics
Bucharest University of Economic Studies

Dissertation Thesis

Coordinator:
Prof. Cristian-Valeriu Toma Ph.D.

Master Student:
Marius-Remus Dumitrel

Bucharest 2026

IT Security Master Program
Department of Economic Informatics and Cybernetics
Faculty of Cybernetics, Statistics and Economic Informatics
Bucharest University of Economic Studies

A Decentralized Authentication Framework for IoT Devices Using Lightweight Blockchain Architecture

Dissertation Thesis

Coordinator:
Prof. Cristian-Valeriu Toma Ph.D.

Master Student:
Marius-Remus Dumitrel

Bucharest 2026

Abstract

Securing the Internet of Things (IoT) currently forces a dangerous compromise. Traditional authentication protocols rely on centralized databases or identity providers, creating massive single points of failure. While blockchain technology solves this by distributing trust, standard public ledgers are simply too computationally heavy and expensive for most limited, battery-powered, legacy edge devices to run.

This research attempts to break that compromise by developing and testing **AuthApp**, a decentralized authentication framework built specifically for constrained environments. Instead of forcing the IoT devices to run heavy consensus algorithms, I shifted the cryptographic burden to a middleware "Fog" layer. Node.js gateways act as intermediaries, allowing the constrained edge devices to authenticate using lightweight Elliptic Curve Cryptography (ECDSA and ECDH) over the fast, UDP-based CoAP protocol. These gateways then securely broker the identities to a permissioned Hyperledger Fabric network running the Raft consensus mechanism.

Crucially, this Multi-Gateway approach achieves true decentralization at the edge. If one gateway goes offline or suffers a cyberattack, devices simply route their CoAP traffic to another available node, completely eliminating the single point of failure. Inside the blockchain layer, a custom-built TypeScript chaincode acts as the ultimate source of truth. The chaincode automatically verifies the ECDSA signatures and logs every identity state transition (Registration, Verification, Revocation) to an immutable ledger, ensuring that no single gateway can unilaterally authorize a rogue device.

To prove this actually works on real-world hardware, I benchmarked the system across a hybrid environment using both x86 Docker containers and a QEMU ARMv7 emulator. The stress tests revealed that the Fog gateways effectively shielded the blockchain from direct traffic spikes. Even when scaling up to 1000 concurrent device requests and pushing the host CPU to 92% utilization, the architecture maintained a stable average latency ranging between 1.7 and 2.2 seconds, without dropping a single packet. By proving that we can decouple identity from centralized servers, without overwhelming legacy microprocessors, this framework provides a practical, deployable path toward Zero-Trust Network Access and GDPR compliance in industrial IoT ecosystems.

Keywords: *IoT Authentication, Hyperledger Fabric, Elliptic Curve Cryptography, Fog Computing, Zero-Trust Architecture, Blockchain Security.*

Statement regarding the originality of the content

I hereby declare that the results presented in this paper are entirely the result of my own creation unless reference is made to the results of other authors. I confirm that any material used from other sources (magazines, books, articles, and Internet sites) is clearly referenced in the paper and is indicated in the bibliographic reference list.

Statement regarding the ethical use of AI

In accordance with the academic integrity guidelines, I hereby declare the extent to which Artificial Intelligence (AI) tools were utilized in the preparation of this Master's thesis.

AI language models (Google Gemini 3.1) were employed strictly as assistive tools to support the writing process. Specifically, AI was utilized for proofreading, grammatical correction, structural formatting, and refining the clarity of the academic language to ensure readability. Furthermore, AI tools were used to assist in IEEE formatting the bibliography and generating some better looking figures for the structural diagrams (e.g. UML charts).

I explicitly declare that the core intellectual property, architectural design, implementation of the **AuthApp** framework (including the Hyperledger Fabric chaincode and Node.js middleware), execution of the emulated Docker tests, and the subsequent analysis of the results are entirely my own original work. No primary research data, experimental results, or core analytical conclusions were generated by AI. Any additional AI assistance (Claude Opus 4.6) with boilerplate code fragments or the frontend User Interface is explicitly mentioned in the source code files as a comment.

I have meticulously reviewed, edited, and validated all content within this document. I assume full and absolute responsibility for the accuracy, originality, and academic integrity of this final thesis.

Submitted in partial fulfillment of the requirements for the degree of Master of Science in Cybersecurity

Table of Contents:

List of Figures:

List of Tables:

List of Abbreviations:

AES	Advanced Encryption Standard
API	Application Programming Interface
ARM	Advanced RISC Machine
ASN.1	Abstract Syntax Notation One
BH	Block Height
CBC	Cipher Block Chaining
CBOR	Concise Binary Object Representation
CLI	Command Line Interface
CoAP	Constrained Application Protocol
CPU	Central Processing Unit
DAG	Directed Acyclic Graph
DDoS	Distributed Denial of Service

DER	Distinguished Encoding Rules
DLT	Distributed Ledger Technology
DTLS	Datagram Transport Layer Security
ECC	Elliptic Curve Cryptography
ECDH	Elliptic Curve Diffie-Hellman
ECDSA	Elliptic Curve Digital Signature Algorithm
GDPR	General Data Protection Regulation
gRPC	gRPC Remote Procedure Calls
HTTP	Hypertext Transfer Protocol
IBC	Inter-Blockchain Communication
IEEE	Institute of Electrical and Electronics Engineers
IIOT	Industrial Internet of Things
IoT	Internet of Things
JSON	JavaScript Object Notation
MQTT	Message Queuing Telemetry Transport
NAT	Network Address Translation
NIST	National Institute of Standards and Technology
OAuth	Open Authorization
OIDC	OpenID Connect
OS	Operating System
PII	Personally Identifiable Information
PKI	Public Key Infrastructure
PQC	Post-Quantum Cryptography
PSK	Pre-Shared Key
QEMU	Quick Emulator
SPOF	Single Point of Failure
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TPM	Trusted Platform Module
TPS	Transactions Per Second
UDP	User Datagram Protocol
ZTNA	Zero-Trust Network Access

Anexa nr. 3.1

DECLARAȚIE

Subsemnatul, DUMITREL MARIUS-REMUS, **absolvent** al programului de studii universitare de masterat "Securitatea informatică" din cadrul Facultății de **Cibernetică, Statistică și Informatică Economică** declar că am utilizat instrumente de inteligență artificială generativă (IAG) în procesul de elaborare a acestei lucrări: "A Decentralized Authentication Framework for IoT Devices Using Lightweight Blockchain Architecture" (lucrare de disertație), exclusiv în scopurile descrise mai jos, cu respectarea principiilor prevăzute în *Ghidul de utilizare a instrumentelor de Inteligență Artificială Generativă (IAG) în cadrul Academiei de Studii Economice din Bucureşti*.

Instrumentul(le) utilizat(e): Google Gemini 3.1

Scopul utilizării: Revizuire lingvistica, prezentarea Bibliografiei in formatul IEEE.

Modul în care a fost utilizat IAG-ul și partea/părțile din lucrare în care acesta a fost utilizat:

Instrumentul a fost utilizat pentru gasirea de sinonime, rafinarea mai lizibila a unor expresii si crearea unui fisier BibTeX care sa permita importarea directa a referintelor din Bibliografie in Zotero, pentru a putea fi folosite in formatul IEEE eficient in Word Office. Pentru revizuirea lingvistica, instrumentul a fost folosit intermintent ca un asistent pentru cautarea unor expresii mai potrivite pentru a imi exprima ideiile prezentate in aceasta lucrare. A fost folosit si pentru a verifica daca organizarea structurala a capitolelor indeplineste cerintele standard ale unei lucrari de dizertatie de Master. Pentru Bibliografie, referintele au fost gasite si analizate individual, instrumentul a fost utilizat doar pentru partea de importare in Zotero (generarea fisierului BibTeX).

Data completă a interacţiunii (anul, luna și ziua): 2026, Mai, 1-31 intermitent.

Instrumentul(le) utilizat(e): Google Gemini Nano Banana

Scopul utilizării: Recrearea unor figuri, intr-un format mai clar, pe baza de architecturi UML deja existente si create individual.

Modul în care a fost utilizat IAG-ul și partea/părțile din lucrare în care acesta a fost utilizat: Pentru ca diagramele produse de UML aveau o marime prea mare si un format neclar in Mermeid UML, a fost utilizat instrumentul Nano Banana pentru a le recrea mai compact, bazat pe codul UML introdus. Instrumentul a fost utilizat pentru Figurile 3 si 5. Acest lucru este precizat si in lucrare.

Data completă a interacţiunii (anul, luna și ziua): 2026, Mai, 22.

Instrumentul(le) utilizat(e): Claude Opus 4.6

Scopul utilizării: Frontend Tailwind CSS si cod boilerplate/asistenta cod.

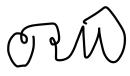
Modul în care a fost utilizat IAG-ul și partea/părțile din lucrare în care acesta a fost utilizat:

Instrumentul a fost utilizat pentru generarea de cod in Tailwind CSS care sa formateze si integreze eficient si prezentabil Interfata Grafica a aplicatiei de Frontend (App.jsx). Instrumentul a fost utilizat si pentru generarea de cod in fabricService.js (alegerea parametrilor pentru stabilirea unei conectiuni corecte) si device.js (asistenta cu functia de conversie a cheilor EC in formatul PEM in Node.js). In general, utilizarea IAG-ului a fost specificata in codul sursa printr-un comentariu specific ce descrie modul de utilizare.

Data completă a interacţiunii (anul, luna și ziua): 2026, Aprilie, 17-27 intermitent.

Sunt conștient că, în cazul constatării falsului voi suporta rigorile legii pentru săvârșirea infracțiunilor comise (fals, uz de fals, fals material în înscrisuri oficiale), conform Art. 326 din Codul penal, cu modificările și completările ulterioare.

Semnătură,



Dată:

18.06.2026

Chapter 1: Introduction

1.1 Context and Problem Statement

In the modern world, new technological advancements like the ability to connect smart physical devices to the internet has fundamentally changed how we interact with them and our environment. From smart TVs, robotic vacuum cleaners, automatic doors or even smart lightbulbs in our homes to industrial sensors and actuators on factory floors, the Internet of Things (IoT) is now everywhere around us. But this rapidly growing new technology hides a massive vulnerability: how we are able to authenticate and secure these devices. Before a humidity or pressure sensor can transmit its data to a cloud service, or before a remote actuator accepts a command to shut off a door or water valve, the network needs to know exactly who is talking. What is the identity of the device sending or receiving the message? This question is the core problem of device authentication.

Right now, the standard for the vast majority of IoT deployments is to handle authentication using different forms of a centralized model. Usually, this means we have a single remote authentication server that holds a massive database of usernames, passwords, or public keys. When a device wakes up, it pings this server to prove its identity through different protocols, as better described by M. A. Khan and K. Salah [1]. While this works well enough for web browsers, it creates a dangerous architectural flaw for physical infrastructure: an absolute single point of failure (SpoF) [2].

If this central server crashes or becomes unusable, for example if an attacker floods it with traffic in a Distributed Denial of Service (DDoS) attack, the entire fleet of devices goes offline. The smart devices relaying on the server as a „trust authority" simply have no other way to verify themselves. Even worse, if a hacker breaches its central database, they instantly compromise and have access to the identities of thousands of devices. Even cases where such attacks don't happen can cause problems, for example when we have a centralized authentication provider like Google or AWS. The tracking and data collection 'habits' of these companies are unfortunately well known; if privacy of your data is a concern, giving away such administrative control over your information to a single entity is simply unacceptable. This naturally makes any such centralized databases a high-value target for attacks, such as distributed denial of service (DDoS), identity spoofing, or replay attacks. In a world where IoT devices control critical infrastructure [3], relying on a single, vulnerable server for network access is a risk we can no longer afford to take.

1.2 Motivation

I chose to tackle the specific problem of IoT device authentication because, while the edge computing landscape is changing fast, most security protocols for IoT are struggling to keep up.

This is also a result of the very heterogeneous (meaning 'different') nature of IoT devices. Since IoT devices have a large pool of providers and their specifications can be very different as a result, there is a clear lack of standards when it comes to most aspects of the IoT „world", as it is also pointed by Stefanescu et al. in [4]. To solve the device identity problem, we need to find a way to authenticate devices securely without bottlenecking them through a single server or save vulnerable stationary identity data in a central database.

Blockchain technology caught my attention early in this research because it completely removes the need for centralized trust [4]. However, public blockchains like Bitcoin [5] or Ethereum [6] were not built for such tiny, resources (CPU, energy, memory, network bandwidth) limited devices. They require massive computational power (like in Bitcoin's Proof of Work consensus) and can charge unpredictable transaction fees (like in Ethereum's Proof of Stake consensus) [7], which makes them very inefficient to use in smart homes, businesses or factories (or even smart cities in general). Such reliance on a digital coin is unnecessary and even undesirable for an enterprise level IoT system.

This research is motivated by the idea that we can bridge this gap. That by using a lightweight, permissioned blockchain, specifically *Hyperledger Fabric* ([8],[9]), we can achieve the security and resilience of decentralized trust without demanding too much from the limited CPUs of legacy IoT hardware. My goal is to build a framework that enables IoT device authentication through this blockchain system, so that trust is distributed rather than hoarded by a single authority. A framework where communication between the blockchain and the devices is handled through *multiple* stateless gateway nodes, in order to achieve true decentralization.

1.3 Thesis Objectives

The main objective of this thesis is to design, implement, and rigorously test a decentralized authentication framework created and customized specifically for realistic resource-constrained IoT devices.

To prove that my architecture is actually viable in the real world, this project focuses on four specific objectives:

1. *Eliminate the Single Point of Failure*: I will build a Multi-Gateway architecture where devices can authenticate against a distributed ledger, while ensuring the network stays up even if an individual gateway goes offline or becomes unavailable.

2. *Push Security to the Edge*: A cryptographic Challenge-Response protocol will be implemented using Elliptic Curve Cryptography [10] (the ECDSA secp256r1 curve will be used), with Application-Layer payload Security (ECDH + AES-256-CBC). These fundamental cryptographic concepts and algorithms are properly presented by Ivan and Toma in their handbook

[11]. This protocol keeps private keys isolated on the devices and protects against eavesdropping, without overwhelming legacy hardware.

3. *Design an on-chain device lifecycle*: I will use a Smart Contract (chaincode in Fabric) to enforce a 4-state finite state machine directly on the blockchain. This contract will automate the identity proving process and ensure that devices are in their proper state for further manipulation (for example a *revoked* device will not be able to authenticate again, but a *suspended* device should be able to). This implementation will also protect us from privilege escalation attack using an API Admin key, together with other security measures for different threats.

4. *Prove Hardware and Network Viability*: I won't just simulate the network in software. A key objective is to run these cryptographic handshakes inside an emulated legacy ARMv7 processor (QEMU) to capture the true latency and hardware constraints that physical edge sensors face. Software emulation will also be used to prove the network's scalability and stability.

1.4 Industry Context and Case Studies

Past events show us the dangers of centralized authentication are not just theoretical. In 2016, the Mirai Botnet provided a terrifying look at what happens when IoT security fails at scale. Mirai infected hundreds of thousands of IP cameras and home routers simply by guessing their default hardcoded credentials [12]. Once infected these devices were weaponized to launch a massive DDoS attack that took down large sections of the internet. If those devices had required a cryptographic decentralized authentication handshake, the attack would have been mathematically impossible, as proven in [13].

This kind of vulnerability is especially critical in domains like the modern automated industry, government institutions or smart healthcare. In a hospital, taking decisions based on a compromised IoT device is not just a nuisance; it is a life-threatening failure. Critical infrastructure that needs to be permanently active and available is a great target for ransomware attacks, as shown by many recent attacks on hospitals and government institutions in the USA, but also in Romania in March 2026 (the ANAF website). By moving trust to the edge and requiring cryptographic proof for every interaction, we can protect these critical sectors from the next iteration of Mirai.

1.5 Research Methodology

To achieve the objectives outlined above, my research was conducted in four distinct phases:

1. *Literature Review*: I have analyzed the current state of IoT authentication and the theoretical boundaries and implementations of lightweight blockchains.

2. *Architectural Design*: Planning my implementation by designing the Multi-Gateway architecture and the Smart Contract State Machine.

3. *Implementation*: Actually coding the Hyperledger Fabric chaincode in TypeScript (the ***DeviceAuthContract*** smart contract) and building the Node.js [14] API middleware and the React [15] frontend for visualization.

4. *Testing and Evaluation:* By deploying the system across a hybrid environment consisting of x86 Docker containers (which are using my computer's processor) and a QEMU emulator (a special Docker container simulating the ARMv7 processor) to benchmark and compare real-world cryptographic overhead, key generation timings and network latency; and stress test the system under various heavy load network conditions.

1.6 Document Structure

The following chapters of this thesis are organized to walk the reader through the problem, the proposed solution, the implementation, and the testing and discussion of the results.

In ***chapter 2***, I present the current state of the art for IoT & blockchain integration, detailing why traditional IoT security fails and how lightweight blockchains offer us a way forward.

Chapter 3 outlines my proposed security architecture, including the specific threat model (how the model handles Sybil and Replay attacks) and the smart contract lifecycle.

Chapter 4 dives into the technical implementation. It breaks down the Hyperledger Fabric setup, the Node.js middleware gateways, and the dual-simulation strategy using Docker and QEMU.

In ***chapter 5***, I will showcase the experimental results. I will analyze the Fabric network configuration, the hard latency numbers, and compare the cryptographic overhead of modern x86 processors against constrained ARM emulators in order to prove this framework's viability for real life implementation. I will also test the framework's scalability and stability, by stress testing the network under various authentication loads.

Chapter 6 provides a critical discussion of the implementation and its results, comparing them to traditional centralized paradigms (like OIDC and Public PKI). It also describes the framework's limitations and analyzes its compliance with modern security regulations like Zero-Trust and GDPR standards.

Finally, in ***chapter 7***, I present the conclusions drawn from my research and outline areas for future development, including the integration of hardware-based root of trust (TPMs) and Post-Quantum Cryptography.

Chapter 2: State of the Art & Theoretical Foundation

2.1 Traditional IoT Security Models

The standard way we secure internet traffic today relies heavily on Public Key Infrastructure (PKI and X.509 certificates) [16] and protocols like Centralized OIDC and OAuth 2.0 [17]. These work very well when used for advanced computers like laptops or smartphones, but they break down when applied to cheap, resource limited IoT devices. These protocols also bring with them a large number of security issues that are unacceptable at an enterprise level, which are well documented in the above referenced papers. Devices running on legacy ARM processors or running off small batteries often cannot handle the heavy computational load required to establish the full TLS handshake most of these protocols rely on [18].

Because full PKI is too heavy for such an environment, developers often take shortcuts. They hardcode passwords or use static API tokens that never change. They then store all these tokens in a massive centralized database [1], which naturally creates a single point of failure. If a hacker manages to extract a static token from just one poorly secured sensor, they can often compromise the entire system, making centralized security solutions very inefficient in distributed environments [19]. This traditional model forces us into a bad compromise: we either burn through the device's battery and processing power with heavy encryption or we rely on weak static passwords stored in a centralized database that is vulnerable to many forms of attacks. The Certificate Revocation Lists (CRLs) PKI relies on to revoke certificates are also known to cause massive bottlenecks. These lists can become very large, in the order of multiple MBs. A memory limited IoT device will simply not be able to download so much information. Even with the use of alternatives like OCSP (Online Certificate Status Protocol), which allows devices to ask a CA (Certificate Authority) about one specific certificate, the authentication still becomes dependent on external internet access to the CA. Many IoT devices at the edge use more restricted wireless networks (e.g. SigFox, LoraWan, Zigbee, etc.), so they simply cannot implement the secure TLS connection to afford such a dependency.

Furthermore, modern identity delegation protocols such as OAuth 2.0 and OpenID Connect (OIDC), while very efficient in web and mobile applications, present significant structural vulnerabilities when ported to enterprise IoT environments. As highlighted by Luo et al. [17], integration platforms relying on OAuth 2.0 inherently depend on centralized Authorization Servers. In a distributed IoT network, routing all authentication requests through a singular, monolithic identity provider creates, once again, a single point of failure. One targeted Distributed Denial of Service (DDoS) attack or an infrastructure outage at the authorization server would instantly sever authentication capabilities for the entire enterprise. Such platforms that rely heavily

on a cloud APIs are also very vulnerable to Cross-App Attacks, an attack in which the attacker intercepts an OAuth authorization code or manipulates the redirect URI. A good example for this was the notorious "Confused Deputy" OIDC Vulnerability between GitHub and AWS [20]. Unlike OIDC, by not relying on such a centralized middle-man, a decentralized authentication framework would simply be immune to Cross-App Attacks.

2.2 Blockchain in IoT

Blockchain technology naturally emerged as the obvious alternative to centralized trust authorities or central databases. By distributing the ledger across multiple nodes, we can completely eliminate such a single point of failure. A hacker cannot take down the network by attacking just one server (an individual node) because every node holds a copy of the truth [21].

Dorri et al. demonstrated that coupling lightweight blockchain ledgers with IoT gateways can eliminate centralized bottlenecks and provide auditable device-to-device communication trails [2]. Christidis and Devetsikiotis further showed that smart contracts can automate security policies directly on the distributed ledger [22]. Stefanescu et al. conducted a systematic literature review covering many recent developments, noting a lack of evaluation standards and a clear trend toward offloading complex cryptographic operations from the IoT node to edge gateways or the ledger itself [4]. Stefanescu also pointed that while decentralization mitigates single points of failure (SpoFs), traditional consesnus mechanisms tend to introduce prohibitive latency costs.

Unsurprisingly, trying to force traditional blockchains into an IoT environment introduced a whole new set of problems. Early research often looked at public blockchains like Bitcoin [5] or Ethereum [6]. These networks use consensus mechanisms like Proof of Work which requires massive amounts of computational power to solve computationally heavy cryptographic puzzles. A Raspberry Pi or a smart thermostat simply does not have the processing power to mine blocks or pay the volatile transaction fees required to interact with these public networks. We need the decentralization of a blockchain, but without such a heavy consensus overhead.

2.3 Hyperledger Fabric Architecture

Among the large variety of options, permissioned ledgers like Hyperledger Fabric created especially for enterprise IoT, seem to address the trade-off of *security vs computational cost* in the most optimal way, whilst maintaining real-world applicability. Unlike public blockchain networks, Fabric is private. You have to be explicitly invited and authorized to join its network [9]. This private nature specific to permissioned blockchains allows for a certain level of trust on which faster and more efficient consensus algorithms can be built. As presented above, most public blockchains use a heavy Byzantine fault tolerant (BFT) consensus, usually dependent on a crypto-currency and transaction fees, where all nodes execute transactions before they can be registered

on the ledger. Fabric has a modular and layered structure, which allows it to separate concerns so that transactions are endorsed only by specific peers and ordered using a Crash fault tolerant (CFT) consensus with a Raft based ordering service [9]. In essence, Hyperledger Fabric behaves like a distributed operating system capable of executing smart contracts (chaincode) in common programming languages (Go, Node.js, Java) with deterministic outcomes and without the need for a native crypto-currency.

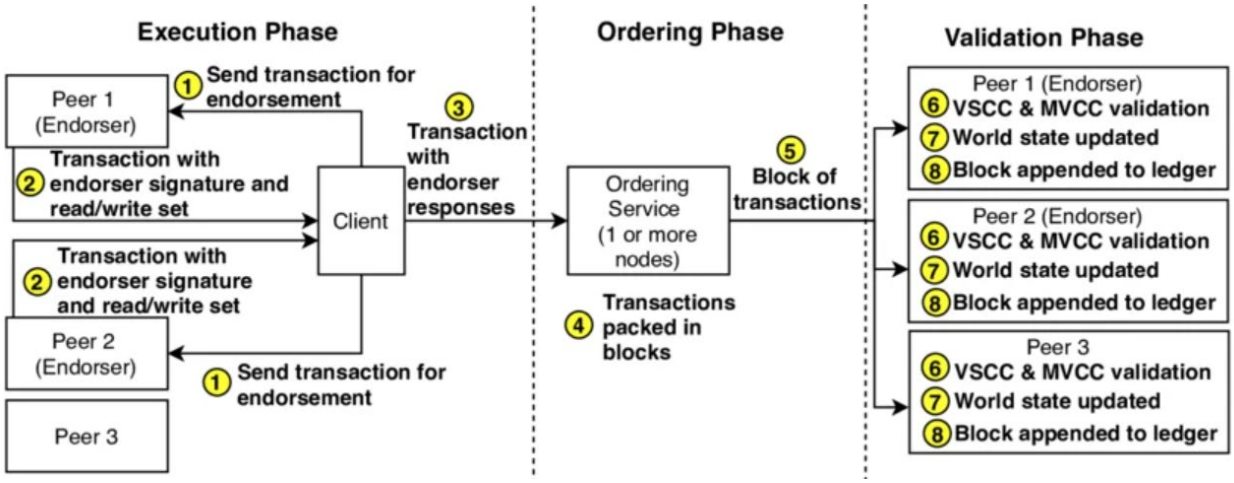


Figure 1. The Execute-Order-Validate transaction flow in Hyperledger Fabric. Adapted from Androulaki et al. [9].

What makes Fabric perfect for this thesis is its unique Execute-Order-Validate architecture, which can be seen in Figure 1. In traditional blockchains, which usually use an Order-Execute architecture, every node has to execute every transaction before it can be added to the block. Fabric separates this process. It allows specific "Endorsing Peers" to execute the transaction first and then sends the results to an "Ordering Service" [8]. For this project I am using the *Raft* [23] consensus mechanism for the ordering service, backed by a distributed *'etcd' cluster* [24]. Raft is crash fault tolerant (CFT) and doesn't require any heavy mining. It simply elects a leader to order the transactions and distributes them to the network. This modular approach means we can achieve incredibly fast transaction finality without asking the IoT devices to do any of the heavy lifting. Gorenflo et al. [25] proved Fabric's efficiency and potential scalability, by implementing "FastFabric", an optimized configuration capable of handling up to 20,000 transactions per second.

2.4 Cryptography on the Edge

Even when using a lightweight blockchain, we still have to prove that a specific device is the one actually sending the message. This requires cryptography. Instead of using older algorithms like RSA, which require massive key sizes to be secure, my implementation utilizes Elliptic Curve Cryptography (ECC) [10] which is better suited for resource constrained devices. With ECC, the devices will have a smaller payload and faster signature generation time.

Specifically, this project uses the ECDSA secp256r1 curve. Elliptic curves provide the exact same level of security as RSA, but with much smaller key sizes [26]. This is a massive advantage when working with the limited memory of legacy ARM processors. To protect the payload data from being intercepted over the air, I also implemented an Application-Layer Security model. The device and the gateway perform an Elliptic Curve Diffie-Hellman (ECDH) asymmetric key exchange to agree on a shared secret, and then use AES-256-CBC to encrypt the actual payload for secure ***Device ←→ Gateway*** communication. This ensures that even if someone is sniffing the wireless network or packet traffic, all they can see is encrypted noise.

Ultimately, what's important is to shield the edge devices from as much heavy cryptographic computation as possible. Therefore, my objective is for all the heavy cryptographic validation operations, like signature verification, to be off-loaded to the blockchain network. This will both create an audit trail and allow the edge devices to focus on the private key encryption process for signature generation and communication.

2.5 Edge and Fog Computing Paradigms

Before further discussing the system's performance, it is important to clearly define where the authentication logic of my project actually lives. As previously discussed, in traditional cloud computing, data is usually generated at the edge and sent all the way to a central control system, like a cloud server, for processing. Despite this cloud-centric model offering virtually unlimited computational resources, it introduces single points of failure, unacceptable network latency, bandwidth bottlenecks, potential availability issues, and heavy data privacy concerns.

To solve the latency and bandwidth limitations, Bonomi et al. introduced the concept of Fog Computing [27]. Fog computing acts as an intermediary layer, pulling the computational power closer to the edge without forcing the tiny IoT devices to do all the work. This paradigm ensures that critical, latency sensitive operations like cryptographic handshakes can be processed locally, without needing to make trips to distant datacenters.

Furthermore, as Dastjerdi et al. [28] emphasize, Fog architectures provide a crucial "shielding" layer for resource-constrained devices. A very resource constrained ARM device simply cannot handle the heavy TCP overhead, the needed persistent connections or the complex consensus protocols required to interact directly with a blockchain network like Hyperledger Fabric. For this, they need something that acts as a router and a translator at the edge gateway level. And that is a Fog Node!

In this thesis, my Node.js Multi-Gateways will function exactly as distributed Fog Nodes. They sit between the vulnerable IoT devices and the heavy Fabric blockchain infrastructure. By doing so, the gateways absorb the immense computational burden of maintaining persistent gRPC

connections, handling the Raft consensus voting, and executing the smart contracts. This allows the end devices to remain incredibly lightweight, communicating strictly via rapid, stateless CoAP over UDP, while the Fog layer securely brokers their identity to the blockchain.

2.6 Comparative Analysis of IoT Blockchain Frameworks

When choosing a blockchain, or better said a *Distributed Ledger Technology* (DLT), for this project, I did not just default to Hyperledger Fabric. To make the best choice, it was essential to examine other recent approaches and implementations, to understand their limitations and architectural trade-offs.

For example, the literature heavily discusses Directed Acyclic Graphs (DAGs) like IOTA as an alternative option, which was created specifically for IoT environments [29]. IOTA uses a *Tangle structure* where each new transaction must verify two previous transactions, allowing it to theoretically scale infinitely without fees. However, while IOTA excels at micro-transactions, it lacks the strict, robust access control required for enterprise authentication [30]. IOTA is a public permissionless network. If we desire to implement device authentication for an enclosed multi-organizational scenario (for example a consortium of companies), we need absolute deterministic control over who is allowed to join the network and who is permanently revoked. Hyperledger Fabric's permissioned channel architecture and its crash-fault tolerant Raft ordering service provide this exact level of enterprise control [29]. It gives us the decentralized trust of a blockchain, but the strict access governance of a private corporate network.

Even though other authors have proposed lightweight blockchain architecture frameworks for edge computing, for example “FogBus” [31] and “LightChain” [32], their implementations still expect considerable computational power from the actual IoT devices. Similarly, Zhao et al. [33] developed an authentication mechanism tailored for the Industrial Internet of Things (IIoT) to minimize cryptographic burdens. Yet, many such custom consensus mechanism implementations rely on protocols that lack the enterprise-grade stability of established frameworks like Hyperledger Fabric, like its crash-fault tolerance and strict cross-organization access control. Bandara et al. [34] in his “Tikiri” framework explored verification awareness in lightweight blockchains to optimize consensus and reduce overhead, but their methodology exposes potential scalability limits under high-density concurrent requests.

In another recent work, Villegas-Ch et al. [19] proposed a lightweight blockchain for authentication in constrained networks. However, their architecture relies heavily on MQTT (Message Queuing Telemetry Transport) for communication between the edge nodes and the central gateway. While MQTT is a lightweight publish-subscribe protocol, it requires a centralized broker to manage the message routing. If this central broker fails, the entire local network segment

goes offline, directly contradicting my goal of ***true decentralization***. In contrast, the approach proposed in this thesis utilizes CoAP (Constrained Application Protocol [35]) and HTTP over a Multi-Gateway architecture. By communicating directly with decentralized Fog gateways without an intermediary broker, we eliminate the single point of failure that plagues MQTT-based systems. Using MQTT for the resource constrained edge communication is also very inefficient, at least a specialized alternative like MQTT-SN (Sensor Networks) should have been used instead.

My approach differs by using the gateway middleware as a translation bridge, which handles the heavy communication to the blockchain. The gateways are stateless and easily replicable, the authentication logic is not dependent on any particular gateway node. The heavy cryptographic computations and storage are managed by the blockchain, at the smart contract (chaincode) level. The private keys never leave the devices and only lightweight payload security is done at the device level.

Chapter 3: Proposed Security Architecture

3.1 High-Level System Architecture

When building any complex network, the "High-Level System Architecture" serves as the macroscopic blueprint. This blueprint will define the major physical and logical components contained in my ecosystem, such as: the edge devices, the intermediate middleware and the backend database. It will also explain how data flows between this project's components. In traditional setups, this architecture usually resembles a centralized hub (like a hub-and-spoke model) with multiple workflows pointing to a single server.

To achieve true decentralization, the architecture proposed in this thesis breaks away from using such a centralized hub format. I designed a Multi-Gateway model where IoT devices are never bound to a single point of failure. Instead, they can freely communicate with multiple independent organization gateways (such as *Org1* or *Org2* from Fabric's test network). From the literature, we can observe these gateways function as Fog nodes. By using them to push the heavy cryptographic workload and consensus logic closer to the edge, we can keep the network decentralized while actively shielding any constrained IoT devices from computational burnout.

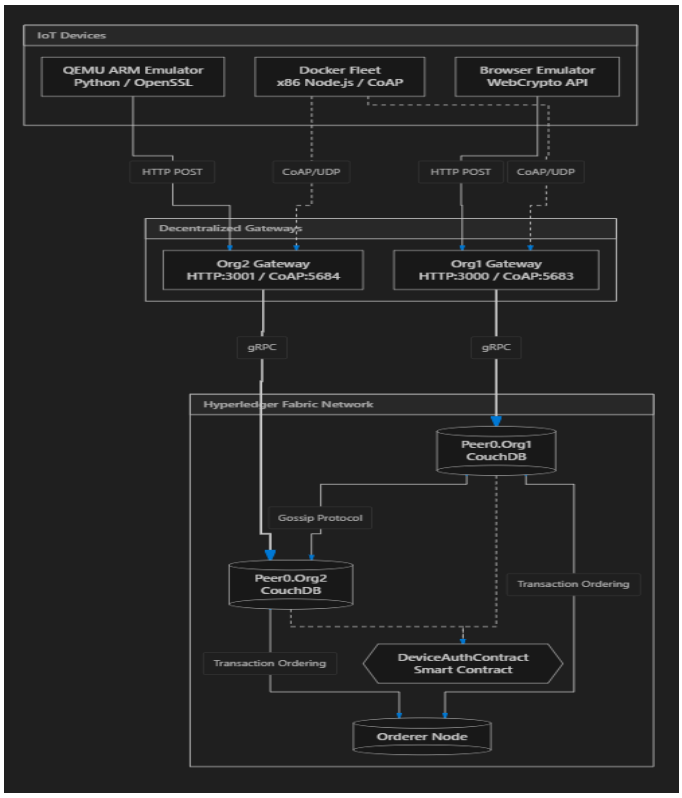


Figure 2. System Architecture Component Diagram

Figure 2 visually maps out this deployment topology. As the diagram illustrates, the physical edge devices are completely separated from the core Fabric ledger. Whether a node is a simple browser emulator or a resource-constrained QEMU ARM instance, it connects to the network via HTTP or CoAP through a distinct organizational gateway. The diagram highlights how these

gateways absorb and package the complex gRPC communication required by the Hyperledger Fabric SDK. Meanwhile, the Fabric network operates securely in the background using CouchDB backed peer nodes and a Raft consensus based Orderer node to maintain a synchronized global state.

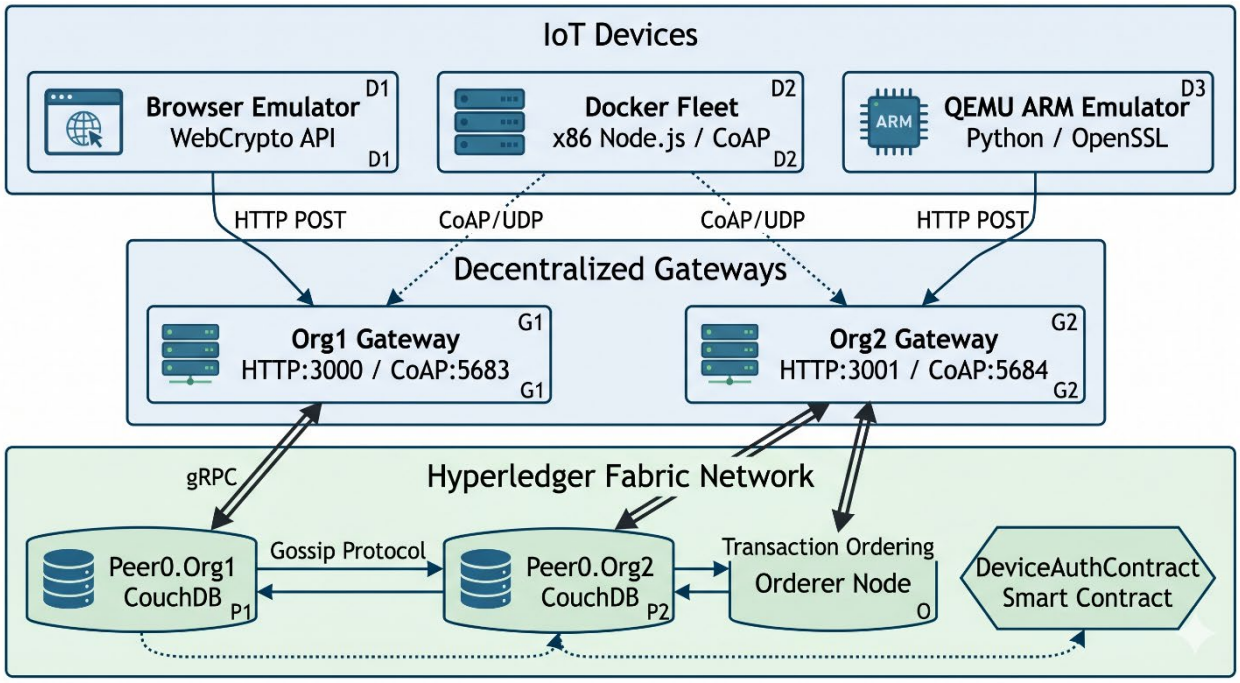


Figure 3. System Architecture Diagram recreated with Gemini

Based on the UML component diagram in Figure 2, I have created a simpler illustration in Figure 3, showing exactly how my proposed framework operates and what device simulation and testing environments have been used during the implementation stage (please ignore the hallucinated *Orderer – Org2* relationship).

Notice the 3 core layers that compose my architecture: the device layer, the gateway (or middleware) layer, and the Fabric blockchain layer. The CouchDB database used by Fabric to store the world state, together with our ***DeviceAuthContract*** smart contract's logic that manages device states on it, could also be considered as a hidden 4th layer. The Frontend Dashboard used for live interaction and visualization acts in many ways as just another device layer component with special permissions. The utility, interaction, and implementation of these layers will be further discussed in the following chapters and sub-chapters. For now, what's important to remember is how the communication flow decouples the heavy cryptographic operations from the resource constrained edge devices. The burden of maintaining the state, validating signatures, and storing identity records is offloaded entirely to the distributed ledger's smart contract (chaincode). This ensures that even if other layers are compromised, the fundamental identities and access privileges of the IoT devices remain anchored on the immutable blockchain ledger.

3.2 Threat Model and Trust Boundaries

We cannot secure any system until we understand exactly how it can be attacked. A "Threat Model" is essentially a structured exercise in paranoia; it tries to identify every potential attack vector an adversary might exploit. Coupled with this is the concept of "Trust Boundaries" which define the exact perimeters where data transitions from an insecure, unverified state into a protected, trusted zone.

In the context of this research, we have to assume the physical IoT device exists in a purely hostile environment. Out in the field, leaf devices like temperature sensors are highly vulnerable to physical tampering and wireless network sniffing. Malicious actors could attempt such side-channel attacks or brute-force operations against the physical edge devices lacking dedicated hardware secure elements. Since hardware security is not currently part of the scope of this thesis, we simply have to assume that some edge devices could become compromised.

Through its architecture, our system is already designed to withstand several common attack patterns targeting IoT deployments. The most obvious one being Distributed Denial-of-Service (DDoS) attacks targeting the centralized credential authorities. Since we do not have such a centralized authority and our devices can authenticate using different organizational gateways, our system is naturally resistant to such forms of attack. Even if a gateway fails, others can simply take its place. However, other forms of attacks still exist, and we must take some special precautions against them, like using a nonce, timestamps and payload security.

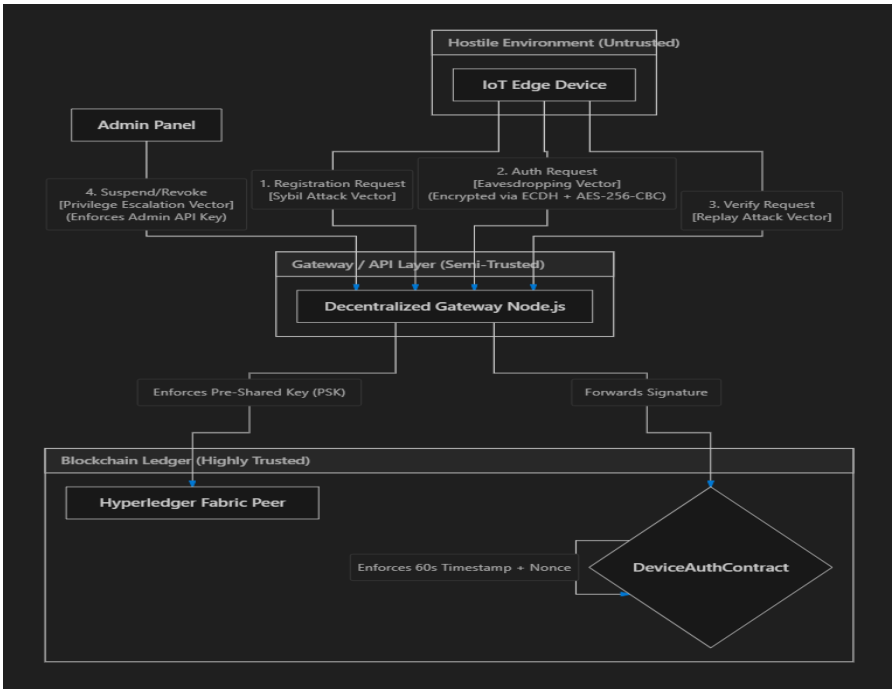


Figure 4. Threat Model and Trust Boundaries flowchart

Figure 4 takes these abstract security concepts and grounds them in a concrete data flow chart. It explicitly partitions the framework into three distinct zones: the Hostile Environment at the edge,

the Semi-Trusted DMZ where our Node.js gateways sit, and the Highly Trusted Blockchain Ledger. The flowchart details the specific perimeter defenses imposed at each boundary. For instance, to stop attackers from flooding the network with fake device identities (a Sybil attack) from the hostile zone, the DMZ middleware boundary strictly enforces a required Pre-Shared Key (PSK). In such a Hostile Environment, devices are vulnerable to physical tampering and network sniffing, which is why the eavesdropping attack vector is addressed using an ECDH and AES-256-CBC encrypted tunnel. The diagram also shows how the blockchain ledger acts as the final arbiter of truth, enforcing a 60-second timestamp window (besides the nonce) to mathematically neutralize Replay attacks. Finally, any privilege escalation attack vectors are handled by requiring an Admin API key for state-altering functions like *suspending* or *revoking* devices.

3.3 Cryptographic Challenge-Response Protocol

Simply connecting a device to a network is not enough, the network needs mathematical proof that this new device is who it claims to be. Inspired by other secure communication protocols, a cryptographic Challenge-Response Protocol is the mechanism I use to verify device identity without ever transmitting its private password over the air. The requested gateway issues a random, single-use "challenge" (a nonce) and the device must generate a valid "response" using its securely stored private key.

To prevent eavesdropping during this exchange, the system utilizes a strict Application-Layer Security tunnel. During the initial registration phase, the device and the gateway agree on an AES-256-CBC shared secret using an Elliptic Curve Diffie-Hellman (ECDH) key exchange.

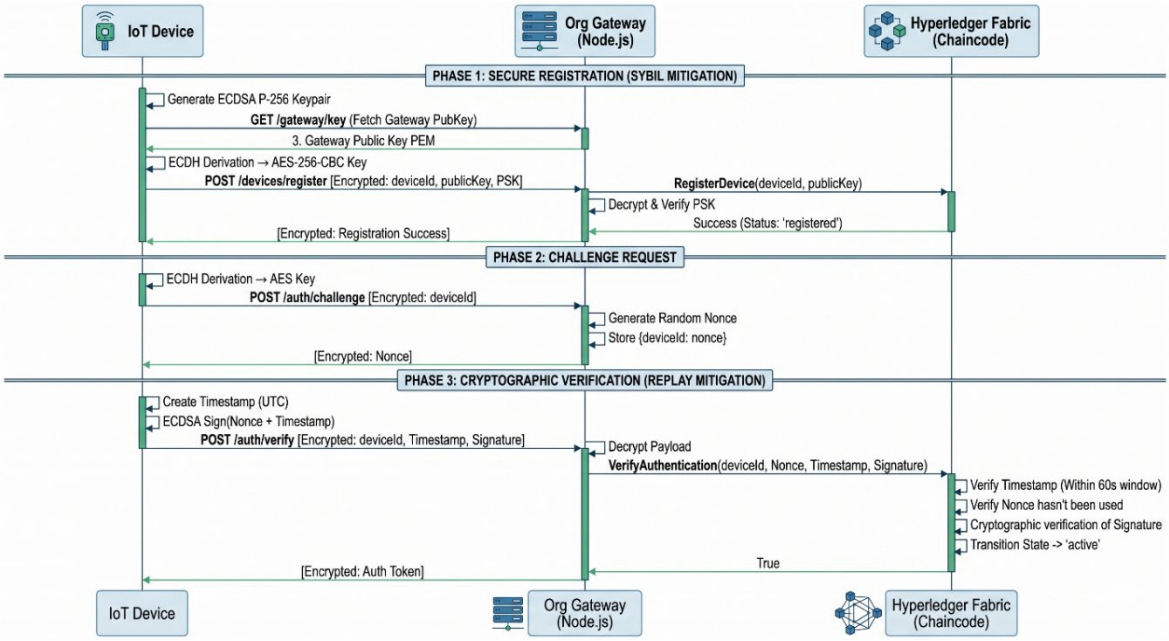


Figure 5. Secure Authentication Sequence Diagram (UML recreated with Gemini)

Because this protocol relies on strict chronological steps, Figure 5 breaks the process down into a sequence diagram. This visualizes the temporal handshake between the IoT device, the Gateway

and the Fabric chaincode. The diagram above walks us through the three critical phases of device lifecycle management implemented in my project.

We start with *Phase 1*, where the device derives its unique AES symmetric key (the shared secret) and registers its public key on the blockchain. Notice how a pre-shared key (PSK), replacing a flashed or manufacturer key in our test scenario, is also necessary for the registration process to succeed. This is my proposed Sybil attack prevention mechanism.

We then move to *Phase 2*, where the gateway issues the *challenge* nonce. To have the best authentication security, this 32-byte nonce needs to be highly randomized, ideally the result of a non-deterministic process. For communication with the gateway, the device can simply use the shared AES key established during the registration process.

Finally, we arrive in *Phase 3*, where the device proves its identity by signing both the nonce and a UTC timestamp with its ECDSA secp256r1 private key. The gateway does not verify this signature, as it stores no public key. It simply decrypts the message, translates it into a gRPC call, and transmits it to the Fabric network, where the entire verification process happens through our deployed chaincode.

By mapping out these interactions, our sequence diagram clearly demonstrates how authentication is achieved without exposing any private device keys to the larger network.

3.4 Chaincode Architecture and Chaincode Device Lifecycle

Once a device's identity is cryptographically verified, it requires further management. A smart contract, also known as chaincode in Fabric, is a distributed autonomous application that can automate security policies directly on the distributed ledger [22]. Thus, we define the "Chaincode Device Lifecycle" as the set of deterministic, coded rules that govern an edge node's identity, operational status and trust, over time. In our blockchain context, a device's identity is not simply created or destroyed, it transitions through various operational states based on strict administrative actions or automated network triggers.

To enforce these rules, my blockchain system relies on the ***DeviceAuthContract***, which functions as a decentralized state machine on Hyperledger Fabric's ledger. Before exploring how a device moves between states, we must understand exactly what data is being stored.

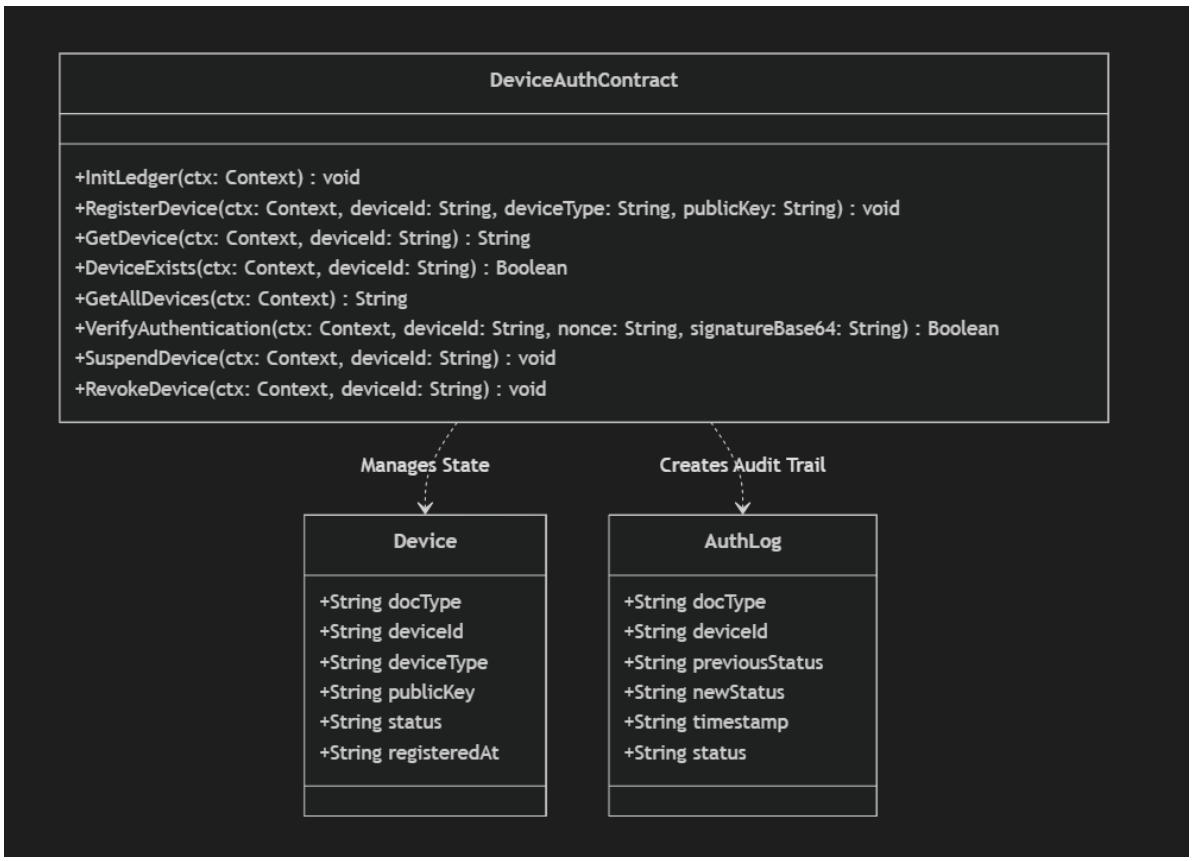


Figure 6. DeviceAuthContract chaincode Class Diagram

Figure 6 presents a UML Class Diagram detailing the specific data structures and methods encapsulated within our ***DeviceAuthContract*** chaincode. My model relies on two primary data structures: ***Device*** and ***AuthLog***. The ***Device*** class represents the asset as it is saved in the CouchDB state database. Note how the ledger does not store any private keys! Instead, it securely stores the *deviceId*, *publicKey*, *deviceType* and the current operational *status*. The ***AuthLog*** class creates a separate, immutable audit trail every time a device changes its status, tracking the *previousStatus*, the *newStatus* and the exact *timestamp* of the transition.

The class diagram above also lists the core executable methods exposed by the ***DeviceAuthContract***. Functions like “InitLedger” and “GetDevice” handle state queries, while “RegisterDevice” and “VerifyAuthentication” manage the automated cryptographic onboarding. Administrative functions like “SuspendDevice” and “RevokeDevice” are reserved exclusively for network governance.

This structural organization of my chaincode directly feeds into how it manages the device's operational lifecycle. Since every state change automatically generates an ***AuthLog*** entry, no single entity can arbitrarily alter a device's permissions without leaving a permanent footprint on the ledger.

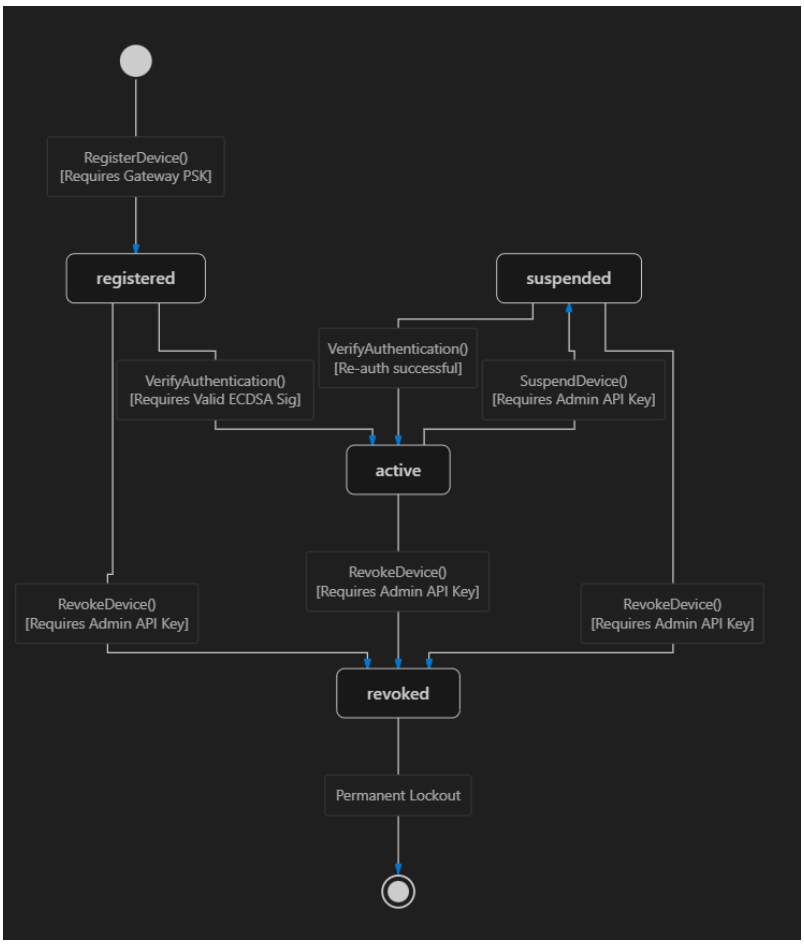


Figure 7. DeviceAuthContract chaincode Lifecycle state machine Diagram

This governance model is presented in Figure 7 through a state machine diagram. It maps out every valid transition any IoT device can make through my chaincode. As shown in the diagram, a device can only begin in a *registered* state, moving to *active* only when it passes the signature verification phase (it mathematically authenticates). However, in reality networks require some additional flexibility. If a network administrator notices an anomaly, like some unusual traffic coming from a particular device, they can trigger the “SuspendDevice” function. The diagram shows how this transitions the device into a *suspended* state, temporarily blocking its network access, while still safely preserving its identity on the ledger. If a device is definitively compromised, it can directly be pushed into a permanent *revoked* state. This diagram ultimately highlights the granular, enterprise-grade access control that permissioned ledgers offer over public alternatives.

Chapter 4: System Implementation

4.1 Hyperledger Fabric Setup

The backbone of this project's decentralized authentication framework is built upon Hyperledger Fabric v2.5.15, using an Arch Linux/amd64 OS [8]. Unlike public ledgers that mostly rely on computationally expensive consensus mechanisms like Proof of Work (PoW), Fabric operates as a permissioned network. As a result, there is a certain organizational level of trust to build upon, as the network is built on the assumption of a multi-organizational environment where the organizations are known to each other. This allows Fabric's network to employ a more efficient crash fault-tolerant Raft (etcdraft [24]) consensus mechanism and remove the need for a central Kafka cluster, which helps in fulfilling this project's decentralization argument.

For this project, a Multi-Organization architecture was deployed featuring 2 organizations: *Org1* and *Org2*.

To ensure the network was optimized for high-throughput IoT traffic, specific parameters were configured within the ***configtx.yaml*** file of the test network. The block-cutting criteria were also explicitly tuned as bellow:

Table 1. Fabric Test-Network configuration

```
# Batch Timeout: The amount of time to wait before creating a batch
  BatchTimeout: 2s
  # Batch Size: Controls the number of messages batched into a block
  BatchSize:
    # Max Message Count: The maximum number of messages to permit in a batch
    MaxMessageCount: 10
    # Absolute Max Bytes: The absolute maximum number of bytes allowed for
    # the serialized messages in a batch.
    AbsoluteMaxBytes: 99 MB
    # Preferred Max Bytes: The preferred maximum number of bytes allowed for
    # the serialized messages in a batch. A message larger than the preferred
    # max bytes will result in a batch larger than preferred max bytes.
    PreferredMaxBytes: 512 KB
  # Organizations is the list of orgs which are defined as participants on
  # the orderer side of the network
```

The parameters in Table 1 mean a new block is "cut" (think like "minted" in Bitcoin) either every 2 seconds or as soon as 10 authentication requests are batched, ensuring rapid transaction finality for authenticating devices. The choice of this configuration, especially the *BatchTimeout* and *MaxMessageCount* parameters, is very impactful on the network's transaction speed and throughput; thus, we will test and analyze it in more depth in the **Results** section.

The core of my authentication logic, the ***DeviceAuthContract*** smart contract showcased in Figure 6, was implemented in TypeScript to leverage Node.js's native JSON handling (commands like *JSON.stringify*), which is described in the broader documentation at [14]. This smart contract implements and manages the finite state machine presented in **Section 3.4**, which enforces the 4 possible device states: *registered*, *active*, *suspended*, and *revoked*.

The lifecycle of an IoT device begins with the ***RegisterDevice()*** transaction. As depicted in the State Machine (Figure 7), the device is initially granted a *registered* status. My implementation explicitly prevents identity hijacking by verifying if the *deviceId* already exists before committing the created ***Device*** structure to the CouchDB state database:

```
@Transaction()
    public async RegisterDevice(ctx: Context, deviceId: string, deviceType:
string, publicKey: string): Promise<void> {
        // 1. Check if the device already exists to prevent identity hijacking
        const exists = await this.DeviceExists(ctx, deviceId);
        if (exists) {
            throw new Error(`The device ${deviceId} is already registered on the
network.`);
        }

        // 2. Construct the device state object
        const device: Device = {
            docType: 'device',
            deviceId,
            deviceType,
            publicKey,
            status: 'registered',
            registeredAt: ctx.stub.getTxID(),
        };

        // 3. Save to the ledger
        // Note: For deterministic sorting in production, standard stringify is
often replaced with deterministic-stringify
        await ctx.stub.putState(deviceId, Buffer.from(JSON.stringify(device)));

        // 4. Emit an event for the Node.js Middleware to catch
        ctx.stub.setEvent('DeviceRegistered', Buffer.from(JSON.stringify({
deviceId, status: 'registered' })));
    }
```

Once registered, the new IoT device must prove its identity via the ***VerifyAuthentication()*** transaction (the entire source code for this core function is available in **Appendix A**). When a device authenticates, the chaincode executes the cryptographic validation natively on the peer. To illustrate this, I present the following chaincode code snippet that enforces a strict 60-second time

window by extracting the deterministic transaction timestamp directly from the ledger state *ctx.stub.getTxTimestamp()*:

```
 // 2.5 Replay Attack Protection (Nonce Cache & Deterministic Time Validation)
        const nonceKey = `NONCE_${nonce}`;
        const nonceExists = await ctx.stub.getState(nonceKey);
        if (nonceExists && nonceExists.length > 0) {
            throw new Error(`Replay Attack Detected: The nonce ${nonce} has
already been used.`);
        }

        // Validate the device's timestamp is within the allowed window (limit:
60 seconds) of the transaction timestamp to prevent replay attacks with old
signatures
        const timestampObj = ctx.stub.getTxTimestamp();
        let txSeconds = 0;
        // The timestamp can be represented in different formats depending on the
environment, so I handle both cases (number or Long)
        if (timestampObj && timestampObj.seconds) {
            txSeconds = (typeof timestampObj.seconds === 'number') ?
timestampObj.seconds : ((timestampObj.seconds as any).low ||
(timestampObj.seconds as any).toNumber());
        }
        const txDate = new Date(txSeconds * 1000);
        const deviceDate = new Date(deviceTimestampStr);
        const timeDiff = Math.abs(txDate.getTime() - deviceDate.getTime());

        // Reject if older than 60 seconds (60,000 ms)
        if (timeDiff > 60000) {
            throw new Error(`Signature Expired: The authentication payload is
outside the valid 60-second time window. txDate: ${txDate}, deviceDate:
${deviceDate}`);
        }
```

If the ECDSA ***secp256r1*** signature is proven valid and the maximum time window is respected, the device transitions to the *active* status. To ensure we have total transparency and auditable state management inside Fabric network, this successful state transition automatically generates an immutable ***AuthLog*** entry on the ledger (as described in Figure 6). This guarantees that administrators can audit the exact lifecycle of any device without having to rely on a centralized database.

In the code bellow, I present this logging process, when a device successfully authenticates and changes its state to *active* on the ledger:

```
 // 5. Audit Trail
        // Generate a log entry on the ledger proving a successful authentication
occurred
        const auditLog = {
```

```
            docType: 'authLog',
            deviceId,
            previousStatus,
            newStatus: 'active',
            timestamp: ctx.stub.getTxID(),  // need this for the date to be
deterministic across peers
            status: 'SUCCESS'
        };
        const logId = `LOG_${deviceId}_${ctx.stub.getTxID()}`;
        await ctx.stub.putState(logId, Buffer.from(JSON.stringify(auditLog)));

        ctx.stub.setEvent('DeviceAuthenticated', Buffer.from(JSON.stringify({
deviceId, previousStatus, newStatus: 'active' })));

        // 6. Mark Nonce as Used to prevent replay
        await ctx.stub.putState(nonceKey, Buffer.from('USED'));
```

To ensure reproducibility and maintain code integrity throughout the development lifecycle, the deployment of the Node.js middleware and the testing of the TypeScript chaincode were automated using continuous integration pipelines via GitHub Actions.

4.2 Dual-Protocol Middleware Gateways

To bridge the gap between the resource-constrained IoT devices and the complex gRPC requirements of the Hyperledger Fabric SDK, I developed a dual-protocol middleware gateway using Node.js. As discussed in **subchapter 3.1** and illustrated in the High-Level System Architecture (Figure 2), this gateway operates as an intermediate Fog node. It buffers the heavy cryptographic processing away from the Edge Layer and handles the secure routing into the Fabric network.

To ensure seamless operation within this project's Multi-Organization architecture, the identity of the gateway is dynamically injected via environment variables. For instance, the ***env.org1*** script bellow configures the Fog node to route transactions exclusively as *Org1*.

```
FABRIC_ORG=org1
FABRIC_MSP_ID=Org1MSP
FABRIC_PEER_ENDPOINT=localhost:7051
FABRIC_PEER_HOST=peer0.org1.example.com
PORT=3000
COAP_PORT=5683
```

A similar script can be used to route transactions to other peer organizations on the network, like *Org2* (the ***env.org2*** script).

Using this configuration, the ***fabricService.js*** script provisions a secure gRPC connection to the peer, utilizing TLS certificates to verify the connection. Once connected, the gateway relies on ***controllers.js*** to enforce application-layer security before any transaction even touches the

blockchain. For example, to prevent Sybil attacks during the registration phase (as modeled in the Threat Model, **subchapter 3.2**), the controller requires a Pre-Shared Key (PSK):

```
        // Validate Pre-Shared Key to prevent unauthorized registration (Sybil
attack mitigation)
        if (!psk || psk !== REGISTRATION_PSK) {
            throw { status: 403, message: 'Registration denied: Invalid or
missing Pre-Shared Key (PSK)' };
        }
        await fabricService.registerDevice(deviceId, deviceType, publicKey);
```

To accommodate different deployment environments, the gateway exposes two distinct API surfaces simultaneously (the full source code for the routing and server initialization can be found in **Appendix B**):

1. HTTP/JSON (Port 3000): A traditional REST API built with node's Express.js framework. This serves as a reliable fallback for environments where UDP traffic is restricted or NAT routing is problematic.

2. CoAP/CBOR (Port 5683): A highly optimized, UDP-based endpoint utilizing the ***coap*** and ***cbor-x*** libraries. The Constrained Application Protocol (CoAP) significantly reduces packet overhead, making it the ideal transport layer for real-world IoT deployments [35].

The middleware's ability to seamlessly decode these binary payloads and route them to the Fabric SDK is demonstrated in the following snippet from the ***coapServer.js*** logic:

```
       let payload = {};
       if (req.payload && req.payload.length > 0) {
            try {
                payload = decode(req.payload);
            } catch (err) {
                console.error('CBOR decode error:', err);
                res.code = '4.00'; // Bad Request
                return res.end(encode({ error: 'Invalid CBOR payload' }));
            }
        }
      // Route to Hyperledger Fabric SDK Controllers
      if (method === 'POST' && url === '/api/v1/auth/verify') {
                result = await controllers.verifyAuthentication(payload);
                res.code = '2.05'; // Content
            }
```

By running these gateways independently for *Org1* and *Org2*, the framework achieves true decentralization. If the *Org1* gateway crashes, devices can simply move to *Org2* and authenticate against the exact same blockchain ledger. This is possible because the gateway is a thin, stateless proxy for the Fabric SDK by design. As an example, the backend was parameterized to use the same codebase to run both organizations, controlled entirely by the presented environment

variables (the 2 *env.org* scripts). On the frontend, a selector toggle was added to target either gateway in real time.

To better view the near stateless nature of the gateways, we can use the following table:

Table 2. Component's State and Gateway dependency

Component	**State location**	**Gateway dependency**
Device identities	Fabric ledger (world state)	None - reads from chain
Cryptographic verification	Chaincode on the peers	None - delegates to chain
Audit trail	Immutable ledger	None - written on chain
Challenge nonces	*challengeStore* (in-memory Map)	***Only temporary state***
Latency metrics	*simulatorLatencies[]* (in-memory)	***Observability only***

Table 2 shows that the only in-memory state is the transient **nonce** challenge (***challengeStore***), which is a short-lived value that exists only between the *challenge* and *verify* calls. There is no shared database between the gateways, the blockchain is our database. It is worth noting that this also means a device's *challenge* and *verify* calls must hit the same gateway instance (session affinity), but this is not a large issue, since our **nonces** are temporary and this time window is very narrow and does not affect the overall decentralization. In production, the **nonce** could also be embedded directly into the Fabric ledger, as an even stronger authentication system "always-up" alternative. Overall, we are protected from single points of failure as the gateways are easily replicable. Even if one gateway is compromised, any other gateway inside the organization can be used to complete the authentication process, as we will show in the Dashboard section (Chapter 4.4).

4.3 The Simulation Strategy (Hardware vs. Software)

Evaluating an IoT framework requires proving both, that the cryptography can run on weak hardware and that the backend can scale to handle thousands of concurrent requests. To address this, I utilized a hybrid simulation strategy:

4.3.1 The QEMU ARM Emulator:

To verify my implementation's feasibility on authentic, legacy hardware architectures, I utilized QEMU (Quick Emulator) [36] to boot a virtualized *qemu-rpi-os-lite:buster-latest* image. This Raspberry Pi OS perfectly emulates the instruction set of a constrained 32-bit Cortex-A series ARM processor [37]. The configuration file used to boot this emulator ***docker-compose-qemu.yml***, together with the bash script ***qemu-setup.sh*** used to automate the deployment and execution of the simulation inside the virtual environment, can be found in **Appendix C**.

Inside the emulator, the client logic is driven by a custom Python script ***device_arm.py***. Due to known limitations with QEMU's SLIRP UDP NAT, which consistently dropped CoAP responses during testing, this client communicates exclusively over the HTTP/JSON gateway endpoint. Furthermore, to capture true hardware constraints, the script offloads the heavy ECDH key agreement and AES-256-CBC encryption to the native ***openssl CLI*** tool. This proves that even older, constrained ARM hardware can successfully negotiate an application-layer secure tunnel and sign authentication nonces without crashing, as demonstrated by the following Python code section:

```
 # 1. Generate ECDH key pair using openssl CLI
        subprocess.check_call(['openssl', 'ecparam', '-name', 'prime256v1', '-
genkey', '-noout', '-out', eph_key_file], stderr=subprocess.DEVNULL)
        subprocess.check_call(['openssl', 'ec', '-in', eph_key_file, '-pubout',
'-out', eph_pub_file], stderr=subprocess.DEVNULL)

        with open(eph_pub_file, 'r') as f:
            eph_pub_pem = f.read()

        # 2. Derive shared secret
        subprocess.check_call(['openssl', 'pkeyutl', '-derive', '-inkey',
eph_key_file, '-peerkey', gw_pub_file, '-out', secret_file],
stderr=subprocess.DEVNULL)

        with open(secret_file, 'rb') as f:
            secret = f.read()

        # 3. Hash secret to get AES key
        import hashlib
        aes_key = hashlib.sha256(secret).digest()
        aes_key_hex = binascii.hexlify(aes_key).decode('utf-8')

        # 4. Generate IV and Encrypt
        iv = os.urandom(16)
        iv_hex = binascii.hexlify(iv).decode('utf-8')

        subprocess.check_call(['openssl', 'enc', '-aes-256-cbc', '-K',
aes_key_hex, '-iv', iv_hex, '-in', payload_file, '-out', encrypted_file],
stderr=subprocess.DEVNULL)
```

As illustrated by the Python snippet above, the process strictly relies on the native OS capabilities. First, the *ecparam* command generates a temporary elliptic curve key pair. Second, *pkeyutl* derives the shared secret by combining the device's temporary private key with the gateway's static public key. Finally, this derived secret is hashed into an ***AES-256 key***, which is used to securely encrypt the payload using Cipher Block Chaining (*'-aes-256-cbc'*) before the

HTTP POST request is dispatched. This code section explicitly proves that the cryptographic overhead modeled in **Chapter 3.2** is entirely feasible on legacy hardware without requiring heavy third-party Python libraries, unlike Node.js which can have huge dependency issues.

4.3.2 The Docker Fleet:

While QEMU proves hardware feasibility, it cannot prove network scalability. To stress-test the Fabric ledger's throughput, I deployed a massive Docker container fleet [38] running on my x86 host architecture. The concurrent deployment configuration file for this fleet ***docker-compose.coap.yml*** is also detailed in **Appendix C**.

The fleet is orchestrated using ***device.js***, a highly concurrent Node.js script. Unlike the ARM emulator, the Docker fleet leverages the CoAP/CBOR protocol by default, firing thousands of lightweight UDP packets at the gateways. To accomplish this, the script natively encodes JSON payloads into raw binary using *cbor-x* before transmitting them over UDP:

```
// Helper to send CoAP POST requests with CBOR payload
function coapPost(endpoint, data) {
    return new Promise((resolve, reject) => {
        const urlString = `${COAP_URL}/${endpoint}`;
        const url = new URL(urlString);
        const req = coap.request({
            pathname: url.pathname,
            host: url.hostname,
            port: url.port || 5683,
            method: 'POST'
        });

        const encodedPayload = encode(data);
        const payloadBytes = encodedPayload.length;

      {
      //------ Response handling omitted for brevity
      }

        req.on('error', reject);
        req.write(encodedPayload);
        req.end();
    });
}
```

As shown in the *coapPost* function, the Node.js Docker fleet avoids heavy HTTP overhead entirely. Before transmission, the *encode(data)* function compactly serializes the JSON request body into a raw binary CBOR format. This binary payload is then dispatched via the native UDP *coap.request()*. This process ensures that the payload size is significantly minimized, which is essential for preserving bandwidth and energy on constrained IoT networks.

To prevent the Fabric orderer from being immediately overwhelmed during a mass startup, the ***device.js*** script incorporates an intelligent exponential back-off and a random initialization delay (0-3 seconds). In the **Results** section, we will show how stress-testing this fleet successfully proves that the dual-gateway architecture can process and commit simultaneous cryptographic authentications at scale without degrading our network's integrity.

4.4 Browser Simulation and Administrative Dashboard

To visualize the real-time state of the decentralized network, a client-side Administrative Dashboard was developed using React and Vite [15].

Beyond purely administrative functions, the dashboard natively simulates the cryptographic behavior of an IoT device, as presented in Figure 3. Because modern browsers cannot access native OS libraries like ***OpenSSL*** (as QEMU does), the React application uses the readily available ***W3C WebCrypto API*** to execute ECDSA *secp256r1* signatures natively within the browser engine. The following code snippet from ***App.jsx*** demonstrates how the browser signs the challenge nonce and timestamp before converting the *IEEE P1363* signature into the *ASN.1 DER* format required by the Fabric chaincode:

```
// 3. Sign the nonce + timestamp with ECDSA SHA-256 using Web Crypto
      addLog(`${deviceId} signing challenge nonce (ECDSA secp256r1)`, 'info');
      const sigStart = performance.now();
      const deviceTimestampStr = new Date().toISOString(); // Added Timestamp
      const payloadString = nonce + deviceTimestampStr;
      const nonceBytes = new TextEncoder().encode(payloadString);
      const signatureP1363 = await window.crypto.subtle.sign(
        { name: 'ECDSA', hash: 'SHA-256' },
        privateKey,
        nonceBytes
      );

      // 4. Convert the IEEE P1363 signature to ASN.1 DER format (what Node's
crypto.createVerify expects)
      const derSignature = ieeeP1363ToDer(signatureP1363);
      const signatureBase64 = arrayBufferToBase64(derSignature.buffer);
      const sigEnd = performance.now();
      const signingMs = Math.round(sigEnd - sigStart);
```

As demonstrated in the code, the simulation constructs the payload by concatenating the challenge nonce with the current ISO timestamp, ensuring the signature adheres to the strict 60 second validity window imposed by my chaincode. The ***window.crypto.subtle.sign*** method then generates an ECDSA signature over the SHA-256 hash of this payload. Crucially, because WebCrypto outputs signatures in the raw ***IEEE P1363*** format, the frontend must manually convert

this binary array into the ***ASN.1 DER*** format. This conversion is strictly required to ensure interoperability with the *Node.js* ***crypto*** library running on the backend middleware and the Hyperledger Fabric chaincode. Also, we can see how we use *signingMs* to compute the signature generation timing for the latency metrics.

The functional implementation of this dashboard, including the: gateway selector, live metric graphs, the authentication log queries, and the administrative device management functions (such as *RevokeDevice, SuspendDevice, etc.*) is best demonstrated visually bellow.

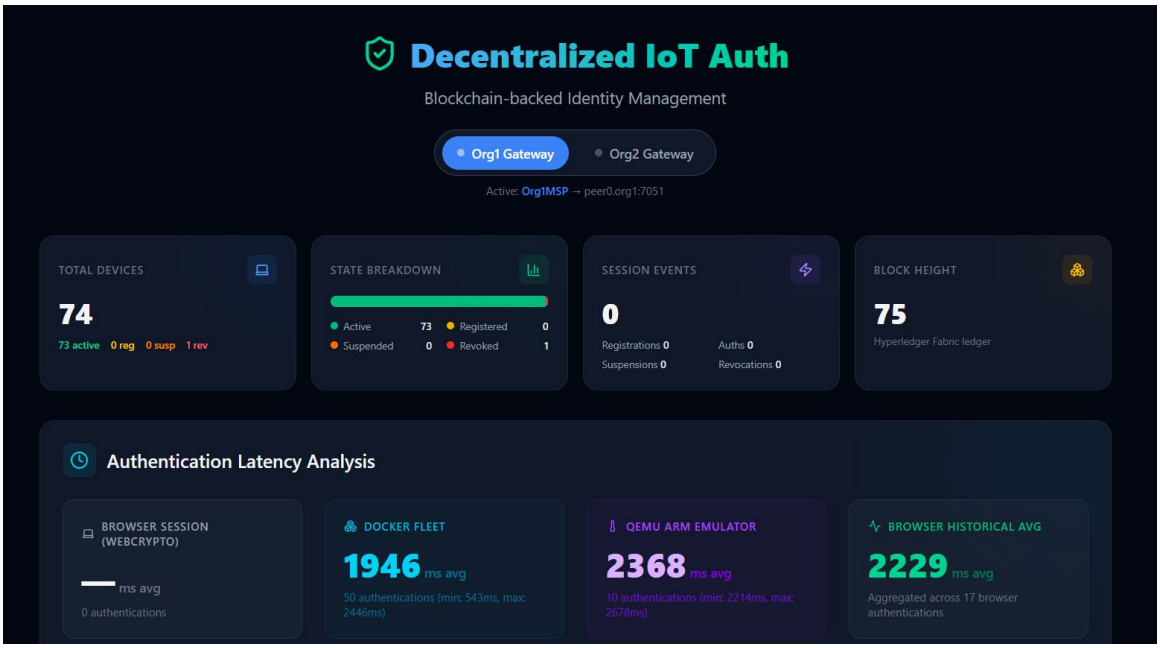


Figure 8. Frontend Administrative Dashboard

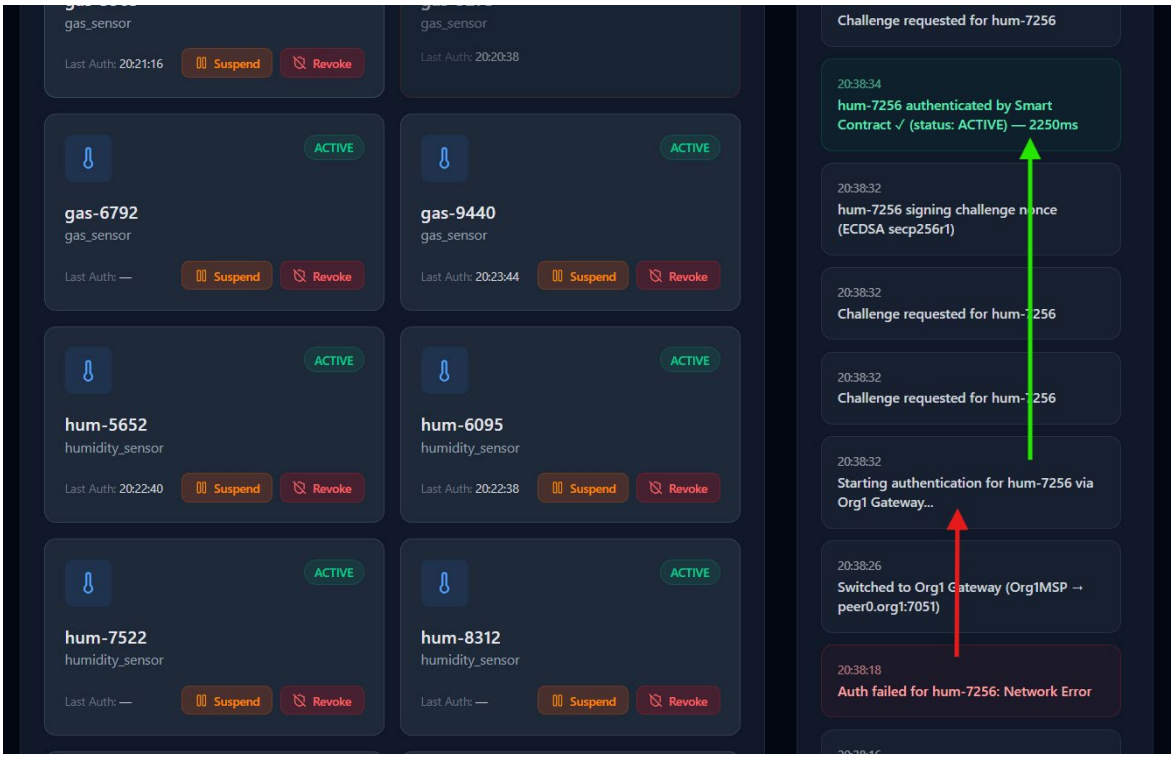


Figure 9. Frontend Device Status and Audit Logs

The dashboard presented in Figure 8 and Figure 9 connects to both the *Org1* and *Org2* middleware gateways, aggregating the ledger's state. It allows administrators to query the immutable ***AuthLog*** trails, view active device connections, and manually trigger state control functions such as *SuspendDevice* or *RevokeDevice*. By utilizing a lightweight frontend framework, the dashboard ensures that security administrators maintain a complete view of the multi-organization network without adding unnecessary overhead to the blockchain peers.

In Figure 8 we observe how the Dashboard acts as a live window into the blockchain, showing us the current blockchain height (BH), the state breakdown of the devices registered on the blockchain, and the average latency metrics of our 3 simulation environments. It also includes other data analysis metrics that will be presented in the next chapter.

In Figure 9 we can see how individual devices are recorded on the blockchain, together with some state altering methods we can use to operate on them. In the Audit trail, on the right side, we also prove our system works as intended when a gateway fails or becomes unavailable. After the *Org2* Gateway has failed, we switch to the *Org1* Gateway and the *hum-7256* device authenticates without a problem. Other core chaincode methods that manipulate devices are also recorded in this log. After testing, *useRef* guards were implemented to protect against multi-click requests.

The Administrative Dashboard also helps us with data collection, as it actively polls the Fabric chaincode for various metrics. The general system implementation included localized phase-timing hooks built directly into the Node.js middleware. By exposing a dedicated */metrics* REST endpoint, the middleware precisely tracks the differences between the initiation of an authentication challenge and the final ledger commitment.

Chapter 5: Experimental Results and Performance Analysis

5.1 Evaluation methodology and Network Configuration

To comprehensively evaluate the performance and scalability of my **AuthApp** framework, a bifurcated simulation strategy was executed across three distinct environments:

1. The ***QEMU ARM Emulator***: A hardware-accurate emulation of a 32-bit Cortex-A series processor running Raspberry Pi OS (Lite), constrained to test legacy IoT cryptographic overhead. To accurately reflect the physical constraints of low-end IoT hardware, a Python 3 execution environment was deployed inside this emulated legacy ARMv7 processor. This provides realistic insight into the cryptographic costs and bottlenecks faced by legacy devices in the IoT field.

2. The ***Docker Fleet (x86):*** A highly concurrent, scalable Node.js container fleet designed to stress-test the Hyperledger Fabric ledger throughput. Utilizing this fleet, deployed within lightweight *node:18-alpine* containers, this environment stress-tested the Fabric ordering service by simulating concurrent CoAP authentication requests without the strict CPU limitations of legacy hardware.

3. The ***Browser Simulator (WebCrypto):*** A modern React-based environment simulating administrative and device interactions natively within the V8 engine. Simulates local x86 execution using the browser's native OS-level cryptography libraries. It serves as the high-performance baseline, representing edge nodes with modern hardware capabilities.

All experiments were connected to my local Hyperledger Fabric network utilizing the Raft consensus protocol, as was presented in **subchapter 4.1**. The network topology and Orderer node configurations remain identical to the system architecture previously established and modeled in **Chapter 3**. The dashboard presented in **subchapter 4.4** will assist me with data collection and analysis. By exposing dedicated REST endpoints and special phase-timing hooks to the middleware, this dashboard precisely tracks the differences between the initiation of an authentication challenge and the final ledger commitment. Furthermore, aggregated metrics were extracted to form the comparative tables and graphs analyzed in the subsequent sections.

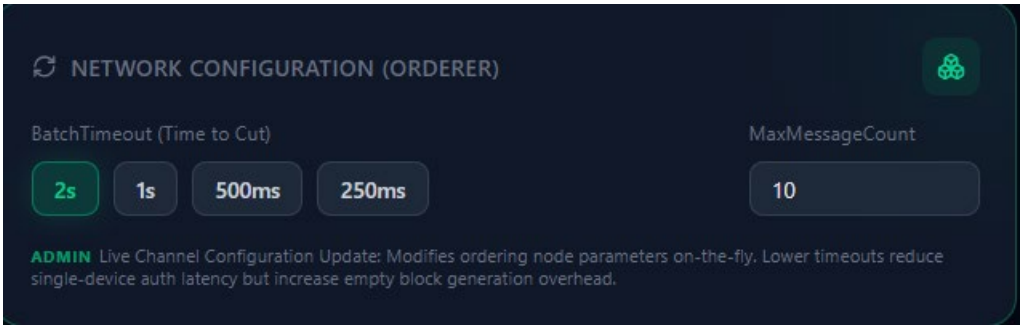


Figure 10. Frontend Network Configuration controller

As we can see in Figure 10, a Network Configuration control panel was added on the Dashboard. This panel allows us to check and modify the Fabric Orderer's configuration (presented in Table 1) on the fly without restarting the network, using a *Channel Configuration Update transaction*, which was implemented with the help of a shell script ***updateBatchTimeout.sh***. Using 2 new endpoints implemented in the Backend API (*GET/POST in /network/ordererConfig*) and protected with the help of the admin API key, this panel was necessary in order to streamline the testing and data gathering, under different Orderer configuration scenarios.

Warning: My UI panel is just a Test Network Administration tool, Orderer configuration controls should never be exposed from the frontend like this, as it means the backend must execute this shell script inside my Docker host. This was simply done to ease my testing process, but should never be done in production.

5.2 Network Configuration Tuning (Ledger Finality)

The Hyperledger Fabric Raft consensus mechanism dictates how rapidly transactions are cut into blocks. As presented before in Figure 1 and **subchapter 2.3**, its transaction processing pipeline relies heavily on the *execute-order-validate* architecture, where the "order" phase is bound by consensus configuration parameters. To determine the optimal enterprise configuration, experiments were conducted by aggressively varying the *BatchTimeout (Time to Cut)* and *MaxMessageCount* parameters.

Table 3. Frontend Cross-Platform Comparison (BatchTimeout = 0.25s, MaxMessageCount = 5)

Authentication Lifecycle — Cross-Platform Comparison

Phase	Browser (WebCrypto) (15)	Docker Fleet (x86) (20)	QEMU ARM Emulator (10)
Key Generation	83 ms	4 ms	191 ms
Registration (Ledger Write)	493 ms	316 ms	779 ms
ECDSA Signing	0 ms	5 ms	123 ms
ECDH + AES Encryption	1 ms	3 ms	582 ms
Auth End-to-End	900 ms	405 ms	1527 ms

Average values per phase across all recorded authentications. Sample sizes shown in parentheses.

In Table 3 we can observe the results of an example test run on the 3 simulation environments, in a scenario where the Fabric orderer's *BatchTimeout* is 0.25s and the *MaxMessageCount* is 5. It is immediately obvious Fabric's configuration (Table 1) has a large impact on our 3 simulation environments, as the *QEMU ARM Emulator* and *Browser Simulator* authenticate IoT devices individually, whilst the *Docker Fleet* environment authenticates multiple IoT devices concurrently.

This means 2 simulation environments (*Browser* and *QEMU*) are negatively affected by a ***2s*** *BatchTimeout* of the Fabric Orderer, as every device's *Registration* and *Authentication* phases

need to wait for this time period before the Orderer cuts their transactions and submits them for validation.

On the other hand, the *Docker Fleet* is not as affected, especially for large number of authentications, as the Orderer simply reaches the maximum number of transactions (*MaxMessageCount*) needed to cut a new block. However, in stress-testing scenarios, we might need the Orderer to have a larger *BatchTimeout* so it does not get overwhelmed by ***micro-minting***. If the Configuration is too aggressive *(i.e. BatchTimeout = 0.25s, MaxMessageCount = 5)*, the Orderer will simply create a very large number of mostly empty blocks (micro-blocks), which means the blockchain's ledger will become unnecessarily large (represented by the Block Height) and each node will have to store this inefficient ledger, increasing disk and CPU overhead costs on the peer nodes.

In conclusion, we seem to have a tradeoff: near-immediate single device authentication vs large network scalability and memory efficiency. Bellow, we'll analyze some of our testing results to prove this assumption.

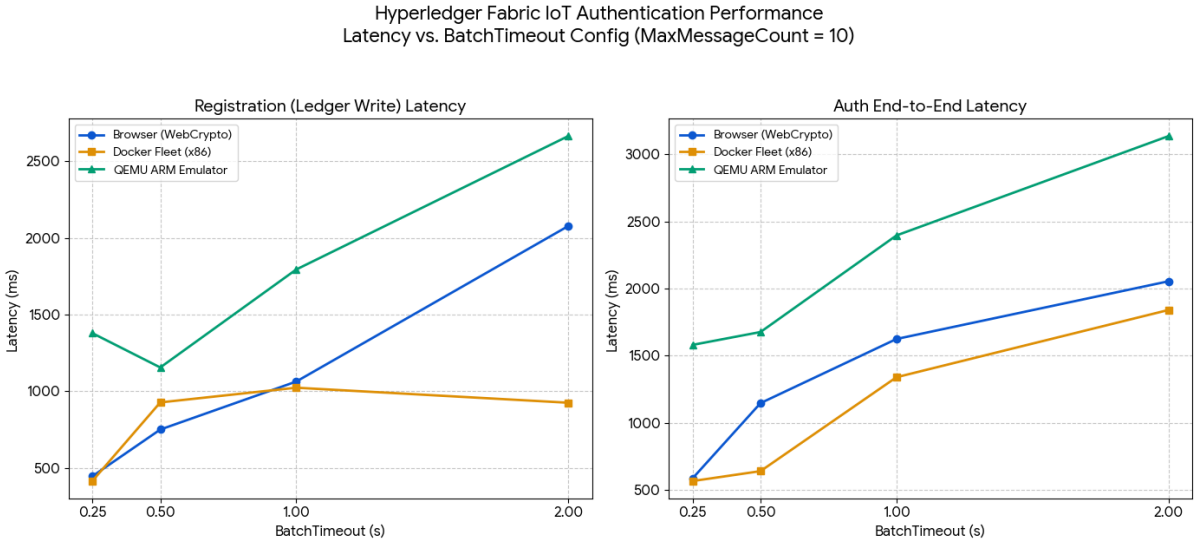


Figure 11. BatchTimeout (Time-to-cut) Configuration Parameter Analysis

The visual trends in Figure 11 clearly demonstrate the higher latency costs of the *QEMU ARM* emulation layer, confirming our implementation's effectiveness in stress-testing constrained IoT environments versus standard *x86 Docker* containers. As expected from our architecture, higher *BatchTimeout* configurations yield roughly proportional latency increases for the *QEMU* and *Browser* simulations, since transactions are forced to wait for the block-cutting timer to expire before being cut into blocks (since the *MaxMessageCount* of 10 is not being saturated). In this case, the *Docker fleet* also appears slightly proportional for End-to-End Authentication, albeit less than the others, but that is a result of doing a small number of device simulations (10-20), which also has the same effect of not always saturating the *MaxMessageCount* to form a new block.

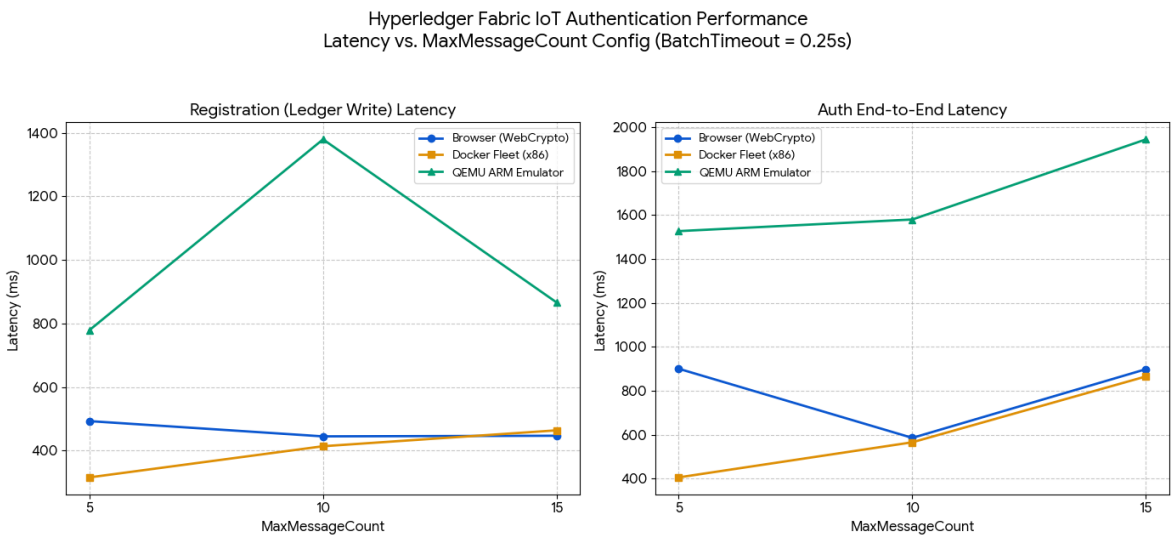


Figure 12. MaxMessageCount Configuration Parameter Analysis

Unlike the linear growth observed with *BatchTimeout*, varying *MaxMessageCount* reveals a non-linear latency behavior in the *Browser* and *QEMU* testing environments (Figure 12). For instance, in the *Browser (WebCrypto)* environment, we observe a distinct latency reduction at a *MaxMessageCount* of 10 for the End-to-End Authentication phase, suggesting optimal block utilization for this specific testing scenario. Logic tells us this is unlikely, as the Browser authenticates devices individually. It should not reach the maximum number of blocks necessary (10) for this parameter to be effective, perhaps this is a result of unintentional concurrent requests in the Browser scenario (like clicking on multiple devices to *Authenticate* at the same time). We can deduce that *MaxMessageCount* parameter does not impact individual authentications severely, meaning this parameter is important mainly for the concurrent *Docker fleet* stress testing scenario, as proven by the linearity depicted in Figure 12. The *QEMU ARM Emulator* continues to demonstrate the highest latency overhead, which is determined by the higher computation cost of its environment.

As also discussed by Gorenflo et al. [25], it is clear *micro-batching* significantly affects overall network throughput and disk I/O. Our findings perfectly align with this literature, generating two operational scenarios of particular interest for computational cost comparisons:

Aggressive Tuning (BatchTimeout = 0.25s, MaxMessageCount = 5): As seen in Table 3, lowering the timeout aggressively forced the *Orderer* to mint micro-blocks rapidly. This resulted in an End-to-End latency drop to roughly ~400 ms for the Docker fleet and ~1.5s for the QEMU emulator. However, the blockchain's *Block Height (BH)* has increased significantly, after running this testing scenario, we had ***BH=74*** (vs *BH=50* for the baseline scenario), a nearly 50% increase in the number of blocks on the ledger.

The Enterprise Baseline (BatchTimeout = 2s, MaxMessageCount = 10): By increasing the timeout, the *Orderer* waits to gather up to 10 transactions into a single block. Consequently, the

End-to-End authentication latency rose to ~1.8s for the Docker fleet and ~3.1s for the QEMU emulation, but the block height ***BH=50*** proved this scenario results in much better memory efficiency on the ledger. This testing environment will be further discussed in **subchapter 5.3**.

With this in mind, we need to also determine the optimal Fabric configuration for the stress testing scenario (i.e. our CoAP *Docker fleet* tests). The following testing results were observed:

Table 4. Consensus Saturation and Resilience (100 Concurrent Devices, CoAP Transport)

BatchTimeout (s)	*MaxMessageCount*	*Average Latency (ms)*	*Block Height (BH)*	*Failed Authentications*
2	5	1645	48	6
2	10	2276	27	0
2	15	2511	24	5
2	20	3704	25	0
1	5	1148	55	3
1	10	1977	34	10

The Cost of ***Aggressive Tuning*** *(MaxMessageCount = 5)*: When forcing the *Orderer* to mint micro-blocks rapidly (e.g. 1s/5 messages or 2s/5 messages), the average latency drops significantly to 1148 ms and 1645 ms respectively (Table 4). The *Orderer* hits this 5 message threshold instantly and cuts a micro-block. However, this triggers a massive expansion in the ledger size (creating 48, respectively 55 blocks for just 100 requests) and induces *CouchDB* state-validation conflicts that cause multiple devices to fail authentication.

The ***Enterprise Baseline*** (*BatchTimeout* = 2s, *MaxMessageCount* = 10): While the average latency rises slightly to ~2.27s (Table 4), the ledger footprint is halved (only 27 blocks). Most importantly, this configuration achieves absolute cryptographic resilience, successfully authenticating 100% of the devices with ***zero failures***. The same resilience is also observed for *MaxMessageCount* = 20, however, by overwhelming the *Orderer* with requests, the transaction latency increases severely to ~3.7s.

To ensure network stability and prevent the *Orderer's* overload during mass concurrency, the ***Baseline*** (*BatchTimeout* = 2s, *MaxMessageCount* = 10) configuration was scientifically selected as the optimal baseline for all subsequent Docker fleet stress testing in **subchapter 5.4**. It provides a balance between speed and memory efficiency that best reflects an enterprise scenario with hundreds (or even thousands) of simultaneously authenticating IoT devices. It is also the default and recommended configuration of the Hyperledger Fabric framework [8].

Note: For all tests performed in this Fabric Configuration section, when the *Block Head (BH)* was computed, the initial 6 Configuration blocks and the extra block created by each configuration change were accounted for! It is important to remember that ***BH*** represents the number of

transactions on the ledger, not the number of devices themselves. Every device status change, for example *Active* → *Suspended*, will request a transaction and impact the ***BH*** as a result. The ***BH*** mainly represents 'activity' on the Fabric Network, not an 'accounting' for the IoT devices, which is mainly done through ***CouchDB*** at peer level. The presented tests were performed without other device state changes, especially due to this consideration.

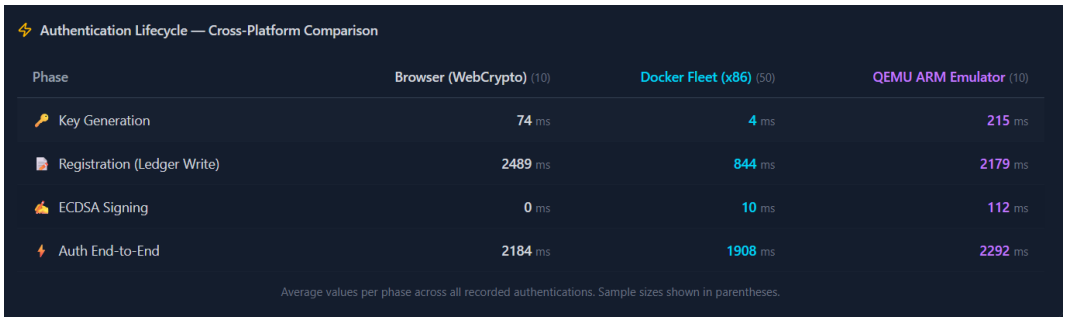
5.3 Cryptographic Overhead Analysis

A critical requirement of the framework is its ability to operate on resource-constrained devices without crashing. To isolate the cryptographic burden, the simulation tracked the execution time of the End-to-End Authentication, Registration, Key Generation, ECDSA Signing, and Application Security (ECDH + AES) Encryption phases across the three testing environments.

The tests were conducted under two application-level security paradigms *(with BatchTimeout = 2s, MaxMessageCount = 10)*:

1. No *Application Security (ECDH + AES)*, relying solely on Fabric's native TLS.

Table 5. No Application Security (ECDH+AES)



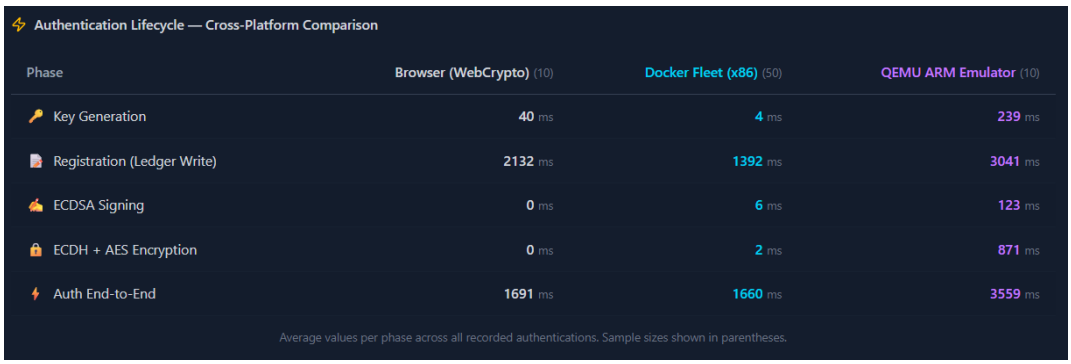
Authentication Lifecycle — Cross-Platform Comparison

Phase	Browser (WebCrypto) (10)	Docker Fleet (x86) (50)	QEMU ARM Emulator (10)
Key Generation	74 ms	4 ms	215 ms
Registration (Ledger Write)	2489 ms	844 ms	2179 ms
ECDSA Signing	0 ms	10 ms	112 ms
Auth End-to-End	2184 ms	1908 ms	2292 ms

Average values per phase across all recorded authentications. Sample sizes shown in parentheses.

2. With *Application Security (ECDH + AES)*, enforcing the custom zero-trust application tunnel between the devices and the gateway.

Table 6. With Application Security (ECDH+AES)



Authentication Lifecycle — Cross-Platform Comparison

Phase	Browser (WebCrypto) (10)	Docker Fleet (x86) (50)	QEMU ARM Emulator (10)
Key Generation	40 ms	4 ms	239 ms
Registration (Ledger Write)	2132 ms	1392 ms	3041 ms
ECDSA Signing	0 ms	6 ms	123 ms
ECDH + AES Encryption	0 ms	2 ms	871 ms
Auth End-to-End	1691 ms	1660 ms	3559 ms

Average values per phase across all recorded authentications. Sample sizes shown in parentheses.

The data in Table 5 and Table 6 reveals that establishing the application-level encryption tunnel adds an average of 0 ms to the *Browser* environment, 2 ms to the *Docker x86 fleet*, and roughly 871 ms to the *QEMU ARM emulator* (in other tests this varied between 500ms and 900ms). This very small overhead on modern architectures, and acceptable <1s total processing time on legacy

hardware, justifies the mandatory inclusion of the *ECDH + AES tunnel* to prevent Man-in-the-Middle (MitM) attacks.

The results in Table 6 demonstrate extreme variance depending on the execution environment:

Browser (WebCrypto): Generating keys takes ~40 ms, while signing and encryption takes practically 0 ms. The browser utilizes highly optimized, compiled C++ engines to execute the *W3C WebCrypto API* natively.

Docker Fleet (Node.js/x86): The Node.js *crypto* library wrappers process the ECDSA and AES operations in an average between 1-6 ms, proving that an x86 Fog Node can easily handle thousands of concurrent cryptographic requests.

QEMU ARM (Legacy Hardware): By offloading the heavy cryptography to the native *openssl CLI*, the 32-bit ARM processor required an average of 239 ms for Key Generation, 123 ms for ECDSA Signing, and 871 ms for ECDH + AES encryption. While significantly slower than modern x86 architectures, an aggregate local processing time of ~1 second (even lower in other tests) proves that the framework is entirely viable for legacy resource limited IoT devices performing periodic authentication without computationally crashing them.

Figure 13. Computational Cost of Key Generation & ECDSA Signing

In a production scenario or deployment, the Application Security (AES+ECDH) should ideally be replaced with proper mTLS (HTTPs) and DTLS (CoAPs) protocols that implement transport security natively, which would heavily reduce the cost for the IoT devices. This unfortunately was not yet possible in this presented framework due to Docker Desktop and WSL connection issues on Windows (the UDP channels from the Docker containers kept dropping before the response from the Middleware could be received) and lack of proper support for DTLS transport in Node.js's *coap* library. With this in mind, the main computational costs of concern for us are the ones presented in Figure 13, the *ECDSA Signing* and *Key Generation*. As it can be seen in Figure 13, for yet another set of tests, the computational costs for our core device level cryptographic

operations remain acceptable. They are very small (under 5ms) for the *Docker Fleet*, the *Browser* simulations maintain only some ~60ms key generation costs, and whilst the QEMU Emulator has significantly larger costs, as expected from constrained IoT devices, their cumulative timing remains under 0.4s consistently. This proves the framework's viability in real life environments.

5.4 Network Scalability and Stress Testing

Beyond raw cryptographic speed, network bandwidth is a precious commodity in constrained IoT environments. This framework's dual-protocol middleware was also analyzed to compare the efficiency of traditional HTTP/JSON against the optimized CoAP/CBOR implementation.

Figure 14. Protocol Efficiency (CoAP vs HTTP)

As it can be seen in Figure 14, over the full authentication lifecycle (*Register*, *Challenge* and *Verify*), the HTTP/JSON payload consumed 1892 bytes. By contrast, the native encoding of JSON payloads into raw binary CBOR transmitted over UDP CoAP, consumed only 1784 bytes. This yields a ***~6% payload size reduction***. While seemingly small per transaction, when extrapolated across the massive Docker fleet simulating hundreds (or even thousands) of concurrent requests, a device load that is very likely in large networks and organizations, this protocol efficiency drastically minimizes bandwidth saturation and reduces the packet processing overhead on the Fog gateways.

In the next part, to validate this framework's architectural resilience, the Docker Fleet was scaled to higher and higher loads of concurrent devices. The objective was to put pressure on the Fabric *Orderer* and see how it behaves under such a heavy set of concurrent requests. The results of these tests were as follows:

Table 7. Network Scalability under Extreme Concurrency (with Baseline Configuration)

Nr. of Devices	*Latency (ms)*	*Throughput TPS (Transactions/sec)*	*Block Height (BH)*	*Failed Authentications*
10	2726	0.96	3	0
20	1837	1.33	5	0

50	1751	1.38	13	0
100	2276	1.35	25	0
200	1819	1.33	50	5
500	1956	1.14	123	0
1000	2064	0.74	280	0

The results in Table 7 are quite outstanding in many ways. First they show the stability of my network, that proves the high scalability of my proposed architecture. Second, they show how the Fabric *Orderer* gets saturated when the number of concurrent requests gets too large. The Block Height (BH) grows proportionally with the number of requests, which is to be expected, since in this scenario almost all transactions will be cut into blocks when they reach the maximum number of transactions needed *MaxMessageCount (10 needed in this Baseline case).* As it was explained in **section 5.2**, the *BatchTimeout* has no contribution in this test scenario, as all blocks get cut before reaching the 2s timeout needed. Finally, the low level of *Failed End-to-End Authentications* is a great surprise, to actually scale to 1000 concurrent devices, whilst succeeding with all *Authentication* requests proves the great resilience and applicability of this framework.

It is also worth pointing out that for 500 concurrent devices the Docker CPU reached values of 78% usage, whilst for 1000 concurrent devices it reached values of 92% usage. This shows that our network was definitely very congested, which did cause TPS drops, but the Fog Middleware successfully queued and processed the requests instead of crashing or dropping packets.

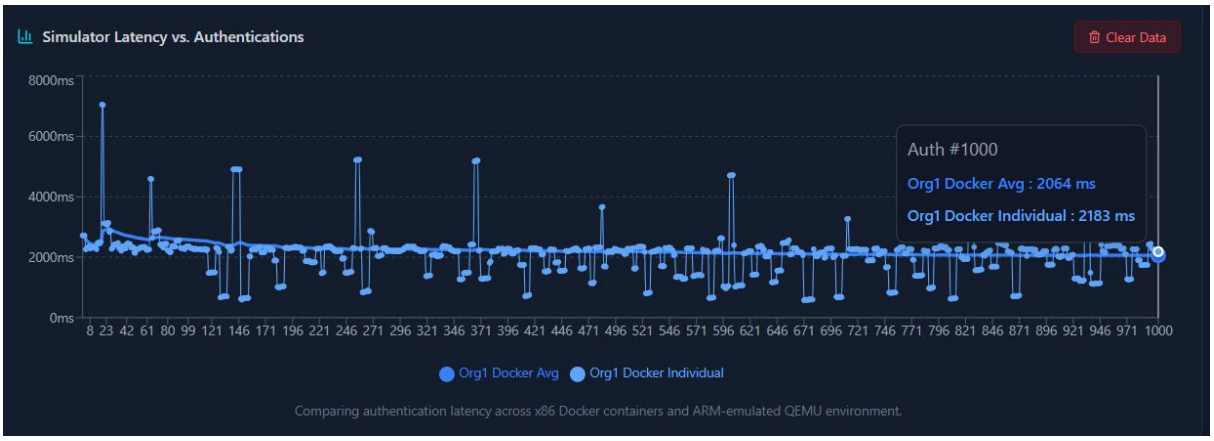


Figure 15. UI Latency for Authenticated devices

To prove the accuracy of the data in Table 7, I provide Figure 15 which is an automatic latency graph my frontend creates for all authenticated devices. In this case, it is the result of the 1000 devices stress test, we can clearly see that all 1000 devices have authenticated successfully.

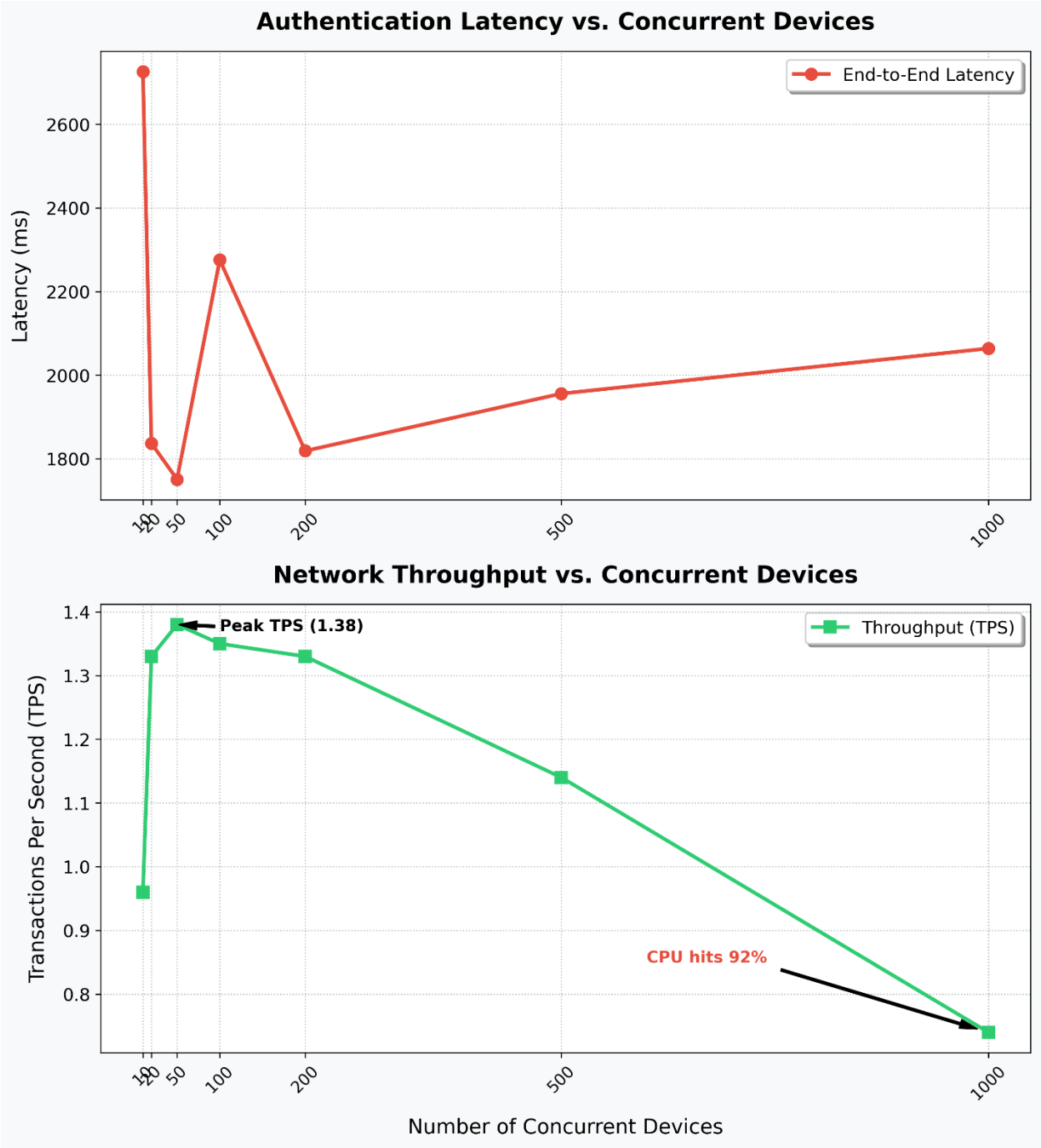


Figure 16. Latency and Throughput vs Concurrent Devices

As depicted in Figure 16 and Table 7, this project's framework exhibits remarkable latency stability. Despite scaling the load by two orders of magnitude, the average end-to-end latency remained incredibly flat, hovering consistently between 1.7s and 2.2s. In the beginning, for only 10 concurrent devices, we see a significantly higher latency of ~2.7s which drops to much lower levels at 20 concurrent devices. This is basically a 'warm-up' effect that is characteristic of JIT-compiled environments like Node.js and connection-pooling optimizations within the Hyperledger Fabric gRPC gateway.

The bottom graph of Figure 16 illustrates the throughput saturation curve, where we can observe that the network reaches a peak TPS of 1.38 at 50 concurrent requests. As the concurrency scales toward 1000 devices, the Docker host CPU utilization reaches 92%, causing the throughput to gracefully degrade to 0.74 TPS. Ultimately, this is a limitation of my computer's processor, which in a 12th Gen Intel(R) Core(TM) i7-1255U 1.70 GHz. For organizations, the *Orderer* service is expected to be on much better servers, where this CPU would allow for far higher throughput. Nonetheless, for the purpose of this paper, the proven throughput and ~2s end-to-end authentication latency at large volumes of 1000 devices is enough to prove both the viability of this system and its superiority to other proposed architectures.

Together with the data in Figure 15, we can observe that rather than dropping packets or crashing under heavy congestion, the dual-gateway Fog Middleware effectively queued the requests, resulting in zero failed authentications during our 1000 devices stress test. This architectural resilience drastically outperforms standard direct-to-blockchain IoT integrations. For comparison, frameworks like FogBus [31] and the lightweight authentication solutions proposed by Fayad et al. [39] often experience exponential latency spikes and packet loss when the concurrent request volume exceeds the underlying consensus finality rate. By deploying the Node.js middleware at the edge, the proposed framework successfully shields the Hyperledger Fabric Orderer from raw UDP traffic bursts, guaranteeing cryptographic integrity and system stability under extreme enterprise loads.

Chapter 6: Discussion

6.1. Comparative Analysis with Traditional Authentication

To better observe the architectural advantages of this project's proposed framework, we should contrast its decentralized edge security and efficiency against the traditional centralized paradigms currently dominating the IoT landscape. The current traditional paradigms were already introduced in **subchapter 2.1**, but now we're going to make a direct comparison between them and our proposed architecture.

Public Key Infrastructure (PKI) and X.509: Traditional enterprise networks rely heavily on Public PKI and X.509 digital certificates to establish trust. However, as demonstrated by Forsby et al. [16], standard X.509 certificates and their associated validation chains (e.g. polling CRLs - Certificate Revocation Lists) result in significant computational and bandwidth overhead. For constrained, remotely operated IoT devices, this overhead is often unmanageable. In contrast, my framework replaces X.509 payload bloat with lightweight ECDSA/ECDH keys verified directly against a localized, immutable ledger, drastically reducing the cryptographic payload while maintaining asymmetric security. By replacing traditional X.509 certificates with raw ECDSA payloads sent over CoAP, we got a highly optimized packet size of just 1784 bytes. Traditional TLS handshakes exchanging full X.509 chains frequently push payload sizes into the several kilobytes range, which easily suffocates narrow-band IoT networks.

Centralized OIDC and OAuth 2.0: While protocols like OAuth 2.0 and OpenID Connect (OIDC) are exceptionally well optimized for consumer smart home environments, they suffer from their inherent centralization vulnerabilities. K. Luo et al. [17] recently highlighted the widespread exploitation of OAuth 2.0 integration platforms, emphasizing the dangers of relying on a singular Authorization Server. In an Industrial IoT (IIoT) context, a centralized server constitutes a catastrophic single point of failure (SpoF); a targeted DDoS attack on the identity provider would sever authentication for the entire industrial grid. Our proposal mitigates this by decentralizing the root of trust across the Hyperledger Fabric nodes, ensuring that the compromise or downtime of a single node does not halt the authentication pipeline.

A highly optimized, centralized web server running OAuth 2.0 or a traditional database can often authenticate a single device in under 100 milliseconds. My framework, heavily bound by the Hyperledger Fabric Raft consensus finality, imposes a hard *latency floor* of roughly 2.0 seconds (as seen in **section 5.4**). For a traditional web developer, adding two seconds of latency just to log in probably sounds unacceptable.

However, in an Industrial IoT context, this is a highly favorable trade-off. What the centralized server gains in raw speed, it loses entirely in resilience. As I have shown in the case study of Mirai,

if a massive botnet targets that central server, its millisecond response time is useless because the server will crash, locking thousands of devices out of the network. My tests proved that by offloading the cryptography to the Fog gateways and pushing the final decision to a distributed ledger, the architecture maintained its ~2.0 second latency even when hammered by 1000 concurrent devices, without dropping a single packet. Ultimately, my ***AuthApp*** framework sacrifices absolute, single-request speed to guarantee absolute network survival and bandwidth efficiency under load, but a more *aggressive* Fabric configuration can be used to significantly increase the individual authentication speed. This can be very useful to smaller size enterprises.

6.2. Architectural Assumptions and Limitations

While the proposed framework demonstrates extreme resilience and great scalability, we should take note of specific architectural assumptions and latency limitations that dictate its operational boundaries. No implementation that is strictly software can ever be without flaw.

To start with, my system fundamentally assumes that the initial ***Pre-Shared Key (PSK)*** required in the *Registration* phase is physically secure. If a malicious actor somehow compromises the device and intercepts the shared secret during the device's first boot, the ledger will permanently register an adversary's public key as the valid one. This means, in production, the ***Pre-Shared keys*** should be generated inside a *Hardware Security Module* (HSM) or *Trusted Execution Environment* (TEE) and be strictly non-extractable.

Secondly, in the current implementation, the ECDH ***gateway public key*** needed to create the AES session key is simply fetched by the device during the *Registration* phase. This was done for simulation convenience and should never happen in production; in a real-world deployment the ***gateway public keys*** should be provided to the devices through out-of-band key provisioning at inner organizational level. They can be flashed directly onto the firmware of the IoT device by the manufacturer together with the ***Pre-Shared Key (PSK)*** during the pre-deployment provisioning phase.

Finally, as it was observed through the tests in **Chapter 5**, the framework is subject to a strict ***latency floor***. As described in earlier chapters, Hyperledger Fabric's execute-order-validate Raft consensus mechanism inherently requires a minimal time to order, broadcast, and validate the blocks, heavily dictated by the *BatchTimeout* configuration. Full end-to-end authentication cannot physically execute faster than the consensus finality rate (typically ~1.5 to 2.0 seconds for the *Baseline* Configuration). Therefore, this framework is clearly not suited for fast real-time systems that require millisecond latency, such as autonomous vehicle braking or emergency medical interventions like CRTs (Cardiac Resynchronization Therapy). It is, instead, designed explicitly

for near real-time telemetry, such as smart grid monitoring, supply chain tracking, and industrial sensor arrays.

6.3. Practical Impact and Regulatory Compliance

Beyond increasing IoT authentication performance, the transition to decentralized architectures carries significant implications for regulatory compliance and enterprise security models.

Zero-Trust Network Access (ZTNA): Modern cybersecurity standards assume that the internal network is fundamentally hostile. Pendli [40] emphasizes that blockchain natively supports Zero-Trust models by decentralizing identity verification. As it was described in earlier chapters, my proposed framework strictly enforces ZTNA via its dual-gateway application-level tunnel. By negotiating a temporary and secure *ECDH + AES-256-CBC* session, the framework ensures that even if the underlying transport (like CoAP over UDP) is compromised or intercepted, the payload remains cryptographically impenetrable. The gateways are treated as transport agnostic, they are basically stateless routers to the Fabric network, the only data they store is the temporary produced nonce, between the *Challenge* and *Verification* phases.

GDPR and Privacy by Design: The European General Data Protection Regulation (GDPR) mandates strict data minimization and the "right to be forgotten", which fundamentally clashes with the immutable nature of public blockchains [41]. This framework's architecture resolves this tension by deploying a ***permissioned*** blockchain (Hyperledger Fabric) and strictly writing only *anonymous identifiers* (Device IDs) and *cryptographic public keys* to the ledger. Because no Personally Identifiable Information (PII) or contextual telemetry data is ever stored on the chain, the framework achieves native compliance with GDPR's "Privacy by Design" principles.

ISO/IEC 27001 Compliance: The ISO 27001 standard requires stringent access control, secure cryptographic asset management, and the continuous auditing of information security systems. My proposed framework acts as a native compliance engine for these requirements. By strictly enforcing ECDSA authentication before granting network access and immutably logging every state transition (*Registration*, *Verification*, *Revocation*) to the distributed ledger, organizations can provide mathematical, tamper-proof audit trails to ISO certifiers, proving that no unauthorized entity has bypassed the authentication perimeter.

Naturally, one might ask: Why is the information on the Fabric blockchain in plaintext, can it not be encrypted? An implementation where only ciphertext would be stored on the ledger is possible, but also rather unnecessary. Fabric, as a permissionless network, has its own unique methods to handle private data: ***Private data*** *&* ***Collections***. The fabric ***Channels*** themselves are only visible to the entities recognized by the *Membership Service Provider* (MSP), so they are enough if the data must only be kept confidential within a set of organizations that are members.

However, there might be a case where we want all channel participants to see a transaction (for auditability), whilst keeping a portion of the data private (like some data that only a specific organization should have access too). This is where Fabric's ***Private data*** feature helps us, as it allows for only a subset of a transaction to be visible on the entire channel (which is basically the blockchain), whilst the rest will be disseminated peer-to-peer to the selected entities in the ***Private Data collection***. This also means, the data will not pass through the *Orderer* nodes, keeping it confidential from the *Ordering Service*. Therefore, the ability to actually visualize certain details of the transaction is actually a feature in the Hyperledger Fabric framework, making encryption on the ledger unnecessary. For further details about Fabric's ***Private data & Collections***, you can consult [8].

Beyond raw latency metrics, the proposed dual-protocol architecture carries significant implications for industrial applications. By decoupling the heavy cryptographic consensus mechanisms from the edge devices and delegating them to the Node.js middleware, organizations can modernize existing "brownfield" deployments. This enables legacy TCP/HTTP devices to achieve decentralized trust without requiring a complete hardware overhaul. Furthermore, replacing centralized credential databases with a mathematically provable, immutable Fabric ledger directly supports emerging global cybersecurity frameworks, such as the **EU Cyber Resilience Act**. The architecture guarantees that all device lifecycle transitions, from registration to revocation, are cryptographically verified and forensically auditable, assisting in strict compliance and risk mitigation without relying on a black-box central authority.

Chapter 7: Conclusions and Future Work

7.1 Conclusion

The main goal of this research was to design and implement a truly decentralized architecture by finding a practical middle ground between the heavy security requirements of modern networks and the severe hardware limitations of IoT devices. Throughout this thesis, I developed and tested the ***AuthApp*** framework, moving away from any centralized databases and trust authorities and instead relying on a permissioned Hyperledger Fabric network to handle device authentication.

What my experimental implementation really showed was that placing stateless gateways as organizational intermediaries is essential. Although there are many proposals for decentralized blockchain systems for IoT devices in literature, most of them use some form of a centralized gateway for Edge←→Blockchain communication, which contradicts the purpose of true decentralization. On the other hand, direct blockchain communication is simply too heavy for embedded IoT devices like small sensors. However, by using CoAP over UDP and handling the heavy gRPC traffic at the Fog layer, the IoT devices only had to focus on the basic ECDSA signing and ECDH key derivation. When my system was pushed to its limits during the stress tests, scaling the load up to 1000 concurrent device requests, the middleware effectively managed and queued the surge in traffic. Even when the Docker host CPU reached ~92% utilization, the average authentication latency stayed consistently between 1.7s and 2.2s, and the system didn't drop a single packet. By implementing a dual-protocol, thus also allowing for legacy HTTP/JSON communication, I have also provided a method for factories or smart cities to integrate their existing, older IoT networks with our framework without having to rip out and replace their physical hardware.

This proves that we don't have to rely on traditional, heavy X.509 certificates or vulnerable centralized servers to secure enterprise or industrial IoT networks. By keeping the cryptography lightweight and shifting the state management to a distributed ledger, the architecture naturally aligns with Zero-Trust principles, ISO 27001 compliance and GDPR data privacy rules, since no personal data is actually stored on-chain.

It is clear from this research, there is still room to optimize the floor latency dictated by the Raft consensus mechanism. For smaller companies, with fewer devices in their IoT fleets, a more aggressive Fabric *Orderer* configuration is clearly preferable, as it severely reduces individual device authentication latency. For more ideas in the direction of Fabric configuration and consensus optimization, we can rely on the “FastFabric” work presented by C. Gorenflo et. al. in [25]. Overall, the presented project and ***AuthApp*** implementation demonstrate that true

decentralized authentication for constrained edge environments is not just something we can conceptualize, but something we can implement in real-world environments.

7.2 Future Work

Looking ahead, there are several ways to build upon the foundation I established with ***AuthApp***. While the current implementation proves that decentralized identity is viable at the edge, the fast-paced changes in IoT hardware and cryptography open up a few clear paths for improvement.

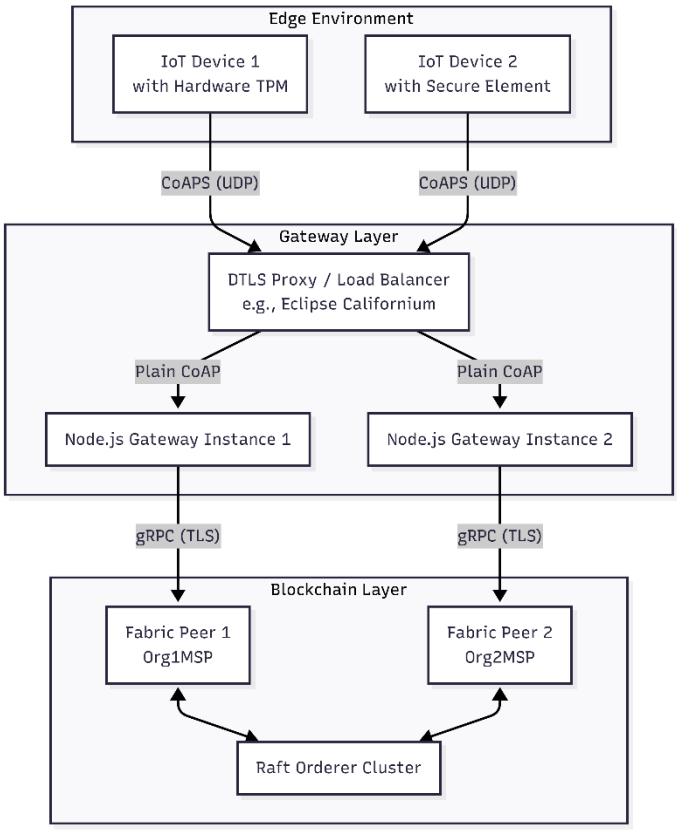


Figure 17. Proposed Future Work Diagram

The UML component diagram in Figure 17 maps out my proposed evolution of the system, it highlights how we can integrate Hardware TPMs/Secure Elements and a DTLS Proxy layer/Load Balancer, which directly addresses the limitations I discussed in ***subchapter 6.2***.

If I were to continue developing this framework, I would choose from four main enhancements:

1. ***Hardware-Based Root of Trust (TPMs):*** A major assumption in my current setup is that the initial Pre-Shared Key (PSK) bootstrapping happens in a physically secure environment, and that the devices store their keys safely in software. In the real world, we need to integrate directly with Trusted Platform Modules (TPMs) or Secure Elements built into the IoT hardware. This physical security layer stops attackers from extracting the private ECDSA keys, even if they steal the device. To take this a step further, organizations should also require manufacturers to flash the gateways' public keys directly into the device firmware during production.

2. ***DTLS Proxy Integration:*** Right now, the system uses an application-layer ECDH derived AES tunnel. To standardize the transport security before the traffic even reaches the Node.js middleware, we could place a Datagram Transport Layer Security (DTLS) proxy, like Nginx or Eclipse Californium, at the edge. This is common in industry and would add the vital layer of encryption needed for the incoming CoAP UDP traffic. However, we must make sure the proxy itself doesn't become a single point of failure. Which means we need a proxy for every gateway!

3. ***Post-Quantum Cryptography (PQC):*** Because I used the secp256r1 elliptic curve, the framework is technically vulnerable to future quantum computing threats (like Shor's algorithm). A necessary next step is to test quantum-resistant algorithms, whether that means lattice-based cryptography or advanced Schnorr signature schemes [26]. Eventually, the smart contracts and device scripts need to transition to NIST-approved Post-Quantum algorithms like CRYSTALS-Kyber or Dilithium. Fortunately, because I built the gateway architecture to be highly modular, we can swap out these heavier cryptographic primitives without having to tear down the underlying Fabric consensus layer.

4. ***Cross-Chain Interoperability:*** I noted earlier that industrial IoT environments are incredibly messy and heterogeneous, there is a huge lack of standards when it comes to IoT manufacturing. Relying entirely on a single permissioned network like Hyperledger Fabric might eventually become a bottleneck as ecosystems expand across different companies. By implementing cross-chain protocols, like Polkadot or Cosmos IBC, the framework wouldn't be locked into just the Hyperledger network. Devices from multiple vendors could authenticate and verify credentials across different administrative boundaries, creating a much larger and more resilient web of trust.

Thank you for taking an interest in this paper! The full source code for the ***AuthApp*** framework can be found, on request, at the address in **Appendix D**.

References

[1] M. A. Khan and K. Salah, "IoT security: Review, blockchain solutions, and open challenges," *Future Gener. Comput. Syst.*, vol. 82, pp. 395–411, 2018.

[2] A. Dorri and others, "Blockchain for IoT security and privacy: The case study of a smart home," in *2017 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*, 2017, pp. 608–613.

[3] O. I. Abiodun and others, "A review on the security of the Internet of Things: Challenges and solutions," *Wirel. Pers. Commun.*, vol. 119, no. 3, pp. 2603–2637, 2021, doi: 10.1007/s11277-021-08348-9.

[4] D. Stefanescu and others, "A Systematic Literature Review of Lightweight Blockchain for IoT," *IEEE Access*, vol. 10, pp. 126233–126252, 2022.

[5] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," Decentralized Business Review, 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[6] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum Project Yellow Paper, 2014. [Online]. Available: https://ethereum.github.io/yellowpaper/paper.pdf

[7] Z. Zheng and others, "An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends," in *2017 IEEE International Congress on Big Data (BigData Congress)*, 2017, pp. 557–564.

[8] Hyperledger Foundation, "Hyperledger Fabric Documentation." 2024. [Online]. Available: https://hyperledger-fabric.readthedocs.io/

[9] E. Androulaki and others, "Hyperledger Fabric: A distributed operating system for permissioned blockchains," in *Proceedings of the Thirteenth EuroSys Conference*, 2018, pp. 1–15.

[10] National Institute of Standards and Technology, "Digital Signature Standard (DSS)," FIPS PUB 186-4, 2013. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.186-4

[11] I. Ivan and C. Toma, *Informatics Security Handbook*, 1st ed. Bucharest, 2006. [Online]. Available: http://www.scribd.com/doc/63750074/Informatics-Security-Handbook-1st-Edition

[12] A. Famera and others, "Analyzing The Mirai IoT Botnet and Its Recent Variants: Satori, Mukashi, Moobot, and Sonic," *IEEE Access*, 2021, [Online]. Available: https://arxiv.org/abs/2508.01909

[13] Z. Ahmed and others, "Protecting IoTs from Mirai Botnet Attacks using Blockchains," in *2019 IEEE 24th International Workshop on Computer Aided Modeling and Design of Communication Links and Networks (CAMAD)*, 2019, pp. 1–6.

[14] Node.js Foundation, "Node.js Crypto Module Documentation." 2024. [Online]. Available: https://nodejs.org/api/crypto.html

[15] Meta Platforms, Inc., "React: The library for web and native user interfaces." 2024. [Online]. Available: https://react.dev/

[16] F. Forsby and others, "Lightweight X.509 Digital Certificates for the Internet of Things," in *Interoperability, Safety and Security in IoT*, vol. 242, Cham: Springer International Publishing, 2018, pp. 123–133. doi: 10.1007/978-3-319-93797-7_14.

[17] K. Luo and others, "Universal Cross-app Attacks: Exploiting and Securing OAuth 2.0 in Integration Platforms," in *Proceedings of the 34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 3221–3238. [Online]. Available: https://www.usenix.org/system/files/usenixsecurity25-luo-kaixuan.pdf

[18] J. P. Ntayagabiri and others, "A Comprehensive Approach to Protocols and Security in Internet of Things Technology," *J. Comput. Theor. Appl.*, vol. 2, no. 3, pp. 324–341, 2024.

[19] W. Villegas-Ch and others, "Lightweight Blockchain for Authentication and Authorization in Resource-Constrained IoT Networks," *IEEE Access*, vol. 13, pp. 48047–48067, 2025.

[20] Cloud Security Alliance AI Safety Initiative, "UNC6426: nx Supply Chain to AWS Admin via OIDC," Mar. 2026. [Online]. Available: https://labs.cloudsecurityalliance.org/research/csa-research-note-oidc-trust-chain-abuse-cloud-takeover-2026/

[21] M. A. Ferrag and others, "Blockchain Technologies for the Internet of Things: Research Issues and Challenges," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2188–2204, 2019.

[22] K. Christidis and M. Devetsikiotis, "Blockchains and Smart Contracts for the Internet of Things," *IEEE Access*, vol. 4, pp. 2292–2303, 2016.

[23] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, Philadelphia, PA, 2014, pp. 305–319. [Online]. Available: https://www.usenix.org/system/files/conference/atc14/atc14-paper-ongaro.pdf

[24] Cloud Native Computing Foundation, “etcd: A distributed, reliable key-value store for the most critical data of a distributed system.” 2024. [Online]. Available: https://etcd.io/
[25] C. Gorenflo and others, “FastFabric: Scaling Hyperledger Fabric to 20,000 Transactions per Second,” *ArXiv Prepr. ArXiv190100910*, 2019, [Online]. Available: https://arxiv.org/abs/1901.00910
[26] J. Na and H. P. In, “S-Auth: Schnorr-enhanced Authentication Scheme for Security and Efficiency in Blockchain Web3.0,” *IEEE Access*, vol. 14, pp. 11455–11472, 2026.
[27] F. Bonomi and others, “Fog Computing and Its Role in the Internet of Things,” in *Proceedings of the first edition of the MCC workshop on Mobile cloud computing*, 2012, pp. 13–16.
[28] A. V. Dastjerdi and others, “Fog Computing: Principles, Architectures, and Applications,” in *Internet of Things: Principles and Paradigms*, 2016, pp. 61–75. [Online]. Available: https://arxiv.org/abs/1601.02752
[29] S. Kumari and J. Thakur, “Performance and Scalability Analysis of Ethereum, Hyperledger Fabric, and IOTA,” *Int. J. Eng. Res. Comput. Sci. Eng. IJERCSE*, vol. 11, no. 12, 2024.
[30] J. P. Mehare and A. K. Gaikwad, “A Comparative Analysis of IoT-Based Blockchain Frameworks for Secure and Scalable Applications,” *Int. J. Intell. Syst. Appl. Eng. IJISAE*, 2023.
[31] S. Tuli and others, “FogBus: A Blockchain-based Lightweight Framework for Edge and Fog Computing,” *J. Syst. Softw.*, vol. 154, pp. 144–166, 2019.
[32] Y. Liu and others, “LightChain: A Lightweight Blockchain System for Industrial Internet of Things,” *IEEE Trans. Ind. Inform.*, vol. 15, no. 6, pp. 3571–3581, 2019.
[33] M. Zhao and others, “Blockchain-Based Lightweight Authentication Mechanisms for Industrial Internet of Things and Information Systems,” *Int. J. Semantic Web Inf. Syst.*, vol. 20, no. 1, pp. 1–30, 2024.
[34] E. Bandara and others, “Tikiri—Towards a lightweight blockchain for IoT,” *Future Gener. Comput. Syst.*, vol. 119, pp. 154–165, 2021.
[35] Z. Shelby and others, “The Constrained Application Protocol (CoAP).” Internet Engineering Task Force (IETF), RFC 7252, 2014. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7252
[36] QEMU, “QEMU Emulator User Documentation.” 2024. [Online]. Available: https://www.qemu.org/docs/master/
[37] ARM Ltd., “Cortex-A Series Processors.” 2024. [Online]. Available: https://developer.arm.com/Processors/
[38] Docker Inc., “Docker Engine overview.” 2024. [Online]. Available: https://docs.docker.com/engine/
[39] A. Fayad and others, “A Blockchain-based Lightweight Authentication Solution for IoT,” in *2020 IEEE International Conference on Communications (ICC)*, 2020, pp. 1–6.
[40] Naveen Reddy Pendli, “Blockchain for Zero-Trust Security Models: A Decentralized Approach to Enterprise Cybersecurity,” *J. Inf. Syst. Eng. Manag.*, vol. 10, no. 33s, pp. 807–813, Mar. 2025, doi: 10.52783/jisem.v10i33s.5651.
[41] R. Belen-Saglam and others, “A systematic literature review of the tension between the GDPR and public blockchain systems,” *Blockchain Res. Appl.*, vol. 4, no. 2, p. 100129, Jun. 2023, doi: 10.1016/j.bcra.2023.100129.

Appendices

Appendix A: Smart Contract Source Code

The following code shows the authentication verification logic (*VerifyAuthentication* transaction) within my Fabric ***DeviceAuthContract***. It enforces the 60-second deterministic timestamp window, checks for nonce reuse to prevent replay attacks, and natively validates the ECDSA **secp256r1** signature:

```
    // VerifyAuthentication is the core function that validates a device's
authentication attempt.
    @Transaction()
    @Returns('boolean')
    public async VerifyAuthentication(ctx: Context, deviceId: string, nonce:
string, deviceTimestampStr: string, signatureBase64: string): Promise<boolean> {
        // 1. Fetch the identity
        const deviceString = await this.GetDevice(ctx, deviceId);
        const device: Device = JSON.parse(deviceString);

        // 2. Validate status : allow registered, active or suspended devices to
authenticate
        const allowedStates = ['registered', 'active', 'suspended'];
        if (!allowedStates.includes(device.status)) {
            throw new Error(`Authentication failed: Device ${deviceId} is marked
as '${device.status}' and cannot be authenticated.`);
        }

        // 2.5 Replay Attack Protection (Nonce Cache & Deterministic Time
Validation)
        const nonceKey = `NONCE_${nonce}`;
        const nonceExists = await ctx.stub.getState(nonceKey);
        if (nonceExists && nonceExists.length > 0) {
            throw new Error(`Replay Attack Detected: The nonce ${nonce} has
already been used.`);
        }

        // Validate the device's timestamp is within the allowed window (limit:
60 seconds) of the transaction timestamp to prevent replay attacks with old
signatures
        const timestampObj = ctx.stub.getTxTimestamp();
        let txSeconds = 0;
        // The timestamp can be represented in different formats depending on the
environment, so I handle both cases (number or Long)
        if (timestampObj && timestampObj.seconds) {
            txSeconds = (typeof timestampObj.seconds === 'number') ?
timestampObj.seconds : ((timestampObj.seconds as any).low ||
(timestampObj.seconds as any).toNumber());
        }
        const txDate = new Date(txSeconds * 1000);
        const deviceDate = new Date(deviceTimestampStr);
```

```
        const timeDiff = Math.abs(txDate.getTime() - deviceDate.getTime());

        // Reject if older than 60 seconds (60,000 ms)
        if (timeDiff > 60000) {
            throw new Error(`Signature Expired: The authentication payload is
outside the valid 60-second time window. txDate: ${txDate}, deviceDate:
${deviceDate}`);
        }

        // 3. Cryptographic Verification
        // We enforce SHA256 hashing to verify the secp256r1 ECDSA signature
        const verify = crypto.createVerify('SHA256');
        verify.update(nonce + deviceTimestampStr);
        verify.end();

        // Pass the stored public key and the device's signature to evaluate the
math
        const isValid = verify.verify(device.publicKey, signatureBase64,
'base64');

        if (!isValid) {
            throw new Error(`Authentication failed: Invalid cryptographic
signature for device ${deviceId}.`);
        }

        // 4. Transition device status to 'active' upon successful verification
        const previousStatus = device.status;
        device.status = 'active';
        await ctx.stub.putState(deviceId, Buffer.from(JSON.stringify(device)));

        // 5. Audit Trail
        // Generate a log entry on the ledger proving a successful authentication
occurred
        const auditLog = {
            docType: 'authLog',
            deviceId,
            previousStatus,
            newStatus: 'active',
            timestamp: ctx.stub.getTxID(),  // need this for the date to be
deterministic across peers
            status: 'SUCCESS'
        };
        const logId = `LOG_${deviceId}_${ctx.stub.getTxID()}`;
        await ctx.stub.putState(logId, Buffer.from(JSON.stringify(auditLog)));

        ctx.stub.setEvent('DeviceAuthenticated', Buffer.from(JSON.stringify({
deviceId, previousStatus, newStatus: 'active' })));

        // 6. Mark Nonce as Used to prevent replay
        await ctx.stub.putState(nonceKey, Buffer.from('USED'));
```

```
        return true;
    }
```

Appendix B: API Routes and Middleware Logic

The code sections presented in this Appendix represent the initialization and routing logic for the dual-protocol middleware gateway.

The ***app.js*** code bellow shows the Express Server Setup for the IoT Authentication Gateway. It sets up the Express Server backend for my Application, by initializing the connection to the Hyperledger Fabric network and defining the API routes for handling frontend requests.

The server listens on the 2 ports (3000 - HTTP, 5683 - COAP) and includes error handling for cases like unhandled promise rejections. By using CORS, I allow the frontend application to communicate with this backend server without cross-origin issues.

```
const express = require('express');
const cors = require('cors');
const { initFabric } = require('./fabricService');
const routes = require('./routes');
const { startCoapServer } = require('./coapServer');

const app = express();
const PORT = process.env.PORT || 3000;
const COAP_PORT = parseInt(process.env.COAP_PORT, 10) || 5683;

app.use(cors());
app.use(express.json());

app.use('/api/v1', routes);

// this async function initializes the connection to the Hyperledger Fabric
network before starting the Express server.
async function startServer() {
    console.log('Initializing Fabric connections...');
    await initFabric();
    require('./cryptoHelper').init();

    // server listens on port 3000 and binds to all network interfaces
    const server = app.listen(PORT, '0.0.0.0', () => {
        console.log(`Backend Server listening on port ${PORT} (0.0.0.0)`);
    });

    // Start the CoAP server for IoT devices
    startCoapServer(COAP_PORT);

    console.log(`Gateway identity: ${process.env.FABRIC_MSP_ID || 'Org1MSP'} |
HTTP :${PORT} | CoAP :${COAP_PORT}`);

    // Keep the process alive — the Fabric gRPC client unrefs internal sockets
```

```
    // which can cause Node's event loop to drain and the process to exit
prematurely
    const keepAlive = setInterval(() => {}, 1 << 30); // ~12 days
    server.on('close', () => clearInterval(keepAlive));
}

process.on('uncaughtException', (err) => {
    console.error('UNCAUGHT EXCEPTION:', err);
});
process.on('unhandledRejection', (reason, promise) => {
    console.error('UNHANDLED REJECTION:', reason);
});
process.on('exit', (code) => {
    console.log(`Process exiting with code ${code}`);
});

startServer();
```

The ***coapServer.js*** code bellow demonstrates how the Node.js middleware intercepts UDP CoAP requests, decodes the CBOR payload and routes the request to the Hyperledger Fabric SDK controllers.

The server listens for requests from IoT devices, processes these requests, and interacts with the Fabric chaincode to manage device credentials and their authentication state. This CoAP server uses CBOR for binary encoding of payloads, which is better than JSON for constrained IoT devices.

```
const coap = require('coap');
const { encode, decode } = require('cbor-x');
const controllers = require('./controllers');

function startCoapServer(port = 5683) {
    const server = coap.createServer();

    server.on('request', async (req, res) => {
        // CoAP request method (for 'POST' and 'GET')
        const method = req.method;
        // The URL path
        const url = req.url.split('?')[0];

        console.log(`[CoAP Server] Received ${method} request for ${url} from
${req.rsinfo.address}:${req.rsinfo.port}`);

        // Note on Security (DTLS / CoAPS):
        // In a production environment, this server should use CoAPS (CoAP over
DTLS)
        // to encrypt the payload and prevent eavesdropping or tampering.
        // Currently, standard Node.js CoAP libraries do not natively support
production-ready DTLS.
```

```
        // Typically, we would use an external proxy like Eclipse Californium or
an Nginx/HAProxy layer
        // to terminate the DTLS connection and forward plain CoAP to this
Node.js instance.

        let payload = {};
        // Decode the CBOR payload if it exists. We expect the payload to be a
CBOR-encoded object containing the necessary fields for each endpoint.
        if (req.payload && req.payload.length > 0) {
            try {
                payload = decode(req.payload);
            } catch (err) {
                console.error('CBOR decode error:', err);
                res.code = '4.00'; // Bad Request
                return res.end(encode({ error: 'Invalid CBOR payload' }));
            }
        }

        // Route the request to the appropriate controller function based on the
method and URL path.
        try {
            let result;
            if (method === 'GET' && url === '/api/v1/gateway/key') {
                result = await controllers.getGatewayKey();
                res.code = '2.05'; // Content
            } else if (method === 'POST' && url === '/api/v1/devices/register') {
                result = await controllers.registerDevice(payload);
                res.code = '2.01'; // Created
            } else if (method === 'POST' && url === '/api/v1/auth/challenge') {
                result = await controllers.requestChallenge(payload);
                res.code = '2.05'; // Content
            } else if (method === 'POST' && url === '/api/v1/auth/verify') {
                result = await controllers.verifyAuthentication(payload);
                res.code = '2.05'; // Content
            } else if (method === 'POST' && url === '/api/v1/metrics/latency') {
                result = controllers.recordLatency(payload);
                res.code = '2.01'; // Created
            } else {
                res.code = '4.04'; // Not Found
                console.log(`[CoAP Server] Endpoint not found: ${url}`);
                return res.end(encode({ error: 'Endpoint not found' }));
            }

            console.log(`[CoAP Server] Sending response for ${url} with code
${res.code}`);
            res.end(encode(result));

        } catch (error) {
            console.error(`CoAP Error [${method} ${url}]:`, error);
            const status = error.status || 500;
```

```
            // Map HTTP status codes to CoAP status codes roughly
            if (status === 400) res.code = '4.00'; // Bad Request
            else if (status === 401) res.code = '4.01'; // Unauthorized
            else if (status === 403) res.code = '4.03'; // Forbidden
            else if (status === 404) res.code = '4.04'; // Not Found
            else res.code = '5.00'; // Internal Server Error

            res.end(encode({ error: error.message || 'Internal error' }));
        }
    });

    server.listen(port, () => {
        console.log(`CoAP Gateway listening on UDP port ${port}`);
    });

    return server;
}
```

Appendix C: Emulation Configuration

The following configuration files define the hardware and software simulation environments used to evaluate the authentication framework.

1. The ***docker-compose-qemu.yml*** (Hardware Emulation)

This file boots a full Raspberry Pi OS inside QEMU to emulate a constrained 32-bit ARM processor.

```
# QEMU ARM Emulator for hardware-accurate IoT authentication latency testing.
# This boots a full Raspberry Pi OS (Lite) inside QEMU, emulating an ARM
processor.

# Usage: docker-compose -f docker-compose-qemu.yml up -d
# Then run the setup script to deploy and execute the simulator: bash qemu-
setup.sh

services:
  qemu-arm:
    image: critoma/qemu-rpi-os-lite:buster-latest
    container_name: qemu-arm-simulator
    ports:
      - "5022:5022"
    restart: unless-stopped
```

2. The **qemu-setup.sh** (Automated SSH Deployment Script)

This bash script automatically resolves the QEMU host IP, establishes an SSH connection to the emulated Raspberry Pi OS, copies the Python simulator, and executes the authentication tests.

```
#!/bin/bash
# Prerequisites:
```

```
#   1. The QEMU container must be running:
#      docker-compose -f docker-compose-qemu.yml up -d
#   2. Wait ~30-60 seconds for the emulated OS to fully boot before running this.
#   3. sshpass must be installed (sudo apt install sshpass) to automate SSH
login.
#
# Usage:
#   bash qemu-setup.sh [number_of_runs]
#   Example: bash qemu-setup.sh 5   (runs 5 sequential authentications)
# ------------------------------------------------------------------------

QEMU_HOST="localhost"
QEMU_PORT="5022"
QEMU_USER="pi"
QEMU_PASS="raspberry"
NUM_RUNS="${1:-1}"

# Determine the host IP that the QEMU VM can use to reach our backend.
# You can override this manually: HOST_IP=192.168.1.100 bash qemu-setup.sh
if [ -z "$HOST_IP" ]; then
    # Method 1: grep /etc/hosts inside the Docker container
    HOST_IP=$(docker exec qemu-arm-simulator grep host.docker.internal /etc/hosts
2>/dev/null | awk '{print $1}')
fi
if [ -z "$HOST_IP" ]; then
    # Method 2: resolve via ping inside the Docker container
    HOST_IP=$(docker exec qemu-arm-simulator sh -c "ping -c1 -W1
host.docker.internal 2>/dev/null" | grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' | head
-1)
fi
if [ -z "$HOST_IP" ]; then
    # Method 3: QEMU default gateway (last resort)
    HOST_IP="10.0.2.2"
    echo "WARNING: Could not auto-detect host IP. Falling back to ${HOST_IP}."
    echo "If this fails, re-run with: HOST_IP=<your-windows-ip> bash qemu-
setup.sh"
fi

SSH_OPTS="-o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o
LogLevel=ERROR -p ${QEMU_PORT}"
SCP_OPTS="-o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o
LogLevel=ERROR -P ${QEMU_PORT}"

echo "========================================="
echo "QEMU ARM Simulator Setup"
echo "========================================="
echo "  Host:     ${QEMU_HOST}:${QEMU_PORT}"
echo "  Host IP:  ${HOST_IP} (for backend access)"
echo "  Runs:     ${NUM_RUNS}"
echo ""
```

```
# Step 1: Wait for SSH to become available
echo "[1/5] Waiting for QEMU SSH to become available..."
MAX_RETRIES=30
RETRY=0
until sshpass -p "${QEMU_PASS}" ssh ${SSH_OPTS} ${QEMU_USER}@${QEMU_HOST} "echo
'SSH ready'" 2>/dev/null; do
    RETRY=$((RETRY + 1))
    if [ $RETRY -ge $MAX_RETRIES ]; then
        echo "ERROR: Could not connect to QEMU VM after ${MAX_RETRIES} attempts."
        echo "Make sure the container is running: docker-compose -f docker-
compose-qemu.yml up -d"
        echo "And wait at least 30-60 seconds for the OS to boot."
        exit 1
    fi
    echo "  Attempt ${RETRY}/${MAX_RETRIES} - waiting 5s..."
    sleep 5
done
echo "  SSH connection established!"
echo ""

# Step 2: Copy the Python simulator to the QEMU VM
echo "[2/4] Copying simulator files to ARM emulator..."
sshpass -p "${QEMU_PASS}" ssh ${SSH_OPTS} ${QEMU_USER}@${QEMU_HOST} "mkdir -p
~/simulator"
sshpass -p "${QEMU_PASS}" scp ${SCP_OPTS} device_arm.py
${QEMU_USER}@${QEMU_HOST}:~/simulator/
echo "  Files copied successfully!"
echo ""

# Step 3: Run the simulator
echo "[3/3] Running ${NUM_RUNS} authentication(s) from ARM emulator..."
echo "  Testing connectivity to backend at ${HOST_IP}:3000..."
sshpass -p "${QEMU_PASS}" ssh ${SSH_OPTS} ${QEMU_USER}@${QEMU_HOST} \
    "curl -s --connect-timeout 5
http://${HOST_IP}:3000/api/v1/network/blockHeight && echo ' Success: Backend
reachable!' || echo ' Failure: Cannot reach backend at ${HOST_IP}:3000'"
echo "========================================="

for i in $(seq 1 ${NUM_RUNS}); do
    echo ""
    echo "--- Run ${i}/${NUM_RUNS} ---"
    sshpass -p "${QEMU_PASS}" ssh ${SSH_OPTS} ${QEMU_USER}@${QEMU_HOST} \
        "cd ~/simulator && SOURCE=qemu API_URL=http://${HOST_IP}:3000/api/v1
python3 device_arm.py 2>&1"

    # I'll enforce a small delay between runs to avoid deviceId collisions
    if [ $i -lt ${NUM_RUNS} ]; then
        sleep 2
    fi
```

```
done

echo ""
echo "========================================="
echo "QEMU ARM simulation complete!"
echo "Check the frontend dashboard for the QEMU latency metrics."
echo "========================================="
```

3. The **docker-compose.coap.yml** (Mass Fleet Simulation)

This file is utilized to scale the Node.js Docker Fleet. It relies on the host's internal gateway to route lightweight UDP (CoAP) packets directly to the Windows-native middleware.

```
# CoAP/CBOR transport variant for the IoT simulator fleet.
# Requires the backend to run on Windows-native Node.js (not WSL)
# so that Docker containers can reach the UDP port directly.
#
# Prerequisites:
#   .\startGateway.ps1 (we need to start the Org1 backend on Windows)
#
# Usage:
#   docker-compose -f docker-compose.coap.yml up --build --scale sensor=5
#
# For Org2 gateway (CoAP port 5684):
#   $env:COAP_GW_PORT="5684" (set up port) -> docker-compose -f docker-
compose.coap.yml up --build --scale sensor=5

services:
  sensor:
    build: .
    extra_hosts:
      - "host.docker.internal:host-gateway"
    environment:
      # CoAP transport (UDP) — requires Windows-native backend (not WSL)
      - COAP_URL=coap://host.docker.internal:${COAP_GW_PORT:-5683}/api/v1
    # DEVICE_ID and DEVICE_TYPE are intentionally NOT set here.
    # Each container will auto-generate a unique ID from its hostname
    # and randomly pick a sensor type, enabling true concurrent scaling.
```

Appendix D: Source Code Repository

The full source code for the **AuthApp** framework is available upon request to the examination committee at: https://github.com/Vincitur/iot-blockchain-auth